

Algorytmy Zaawansowane

Wersja egzaminacyjna

Opracowane na podstawie wykładów Michała Tuczyńskiego

Okrojone do materiału obowiązującego na egzaminie

Spis treści

1	Dziel i zwyciężaj	2
1.1	Znajdywanie pary najbliższych punktów	2
2	Programowanie dynamiczne	5
2.1	Algorytm mnożenia macierzy	5
2.2	Problem mnożenia ciągu macierzy	6
2.3	Problem najdłuższego wspólnego podciągu	11
3	Algorytmy zachłanne	14
3.1	Kody Huffmana	16
3.2	Algorytm Huffmana	17
3.3	Dowód optymalności algorytmu Huffmana	20
3.4	Szeregowanie zadań	20
3.5	Matroidy	22
3.6	Problem szeregowania zadań	23
4	Algorytmy aproksymacyjne	25
4.1	Problem pokrycia wierzchołkowego	26
4.2	Problem pokrycia zbioru	28
4.3	Schematy aproksymacyjne	30
4.4	Problem sumy podzbiorów	31
4.5	Problem sumy podzbioru	33
5	Skojarzenia w grafach	34

Dziel i zwyciężaj

Algorytm typu *dziel i zwyciężaj* składa się z trzech części

1. *Dziel* – problem jest dzielony na mniejsze (zwykle rozłączne) podproblemy,
2. *Zwyciężaj* – podproblemy są rozwiązywane rekurencyjnie,
3. *Połącz* – rozwiązanie całego problemu jest konstruowane z rozwiązań podproblemów.

Złożoność opisuje równanie rekurencyjne $T(n) = a \cdot T\left(\frac{n}{b}\right) + f(n)$, które rozwiązujemy uniwersalnym twierdzeniem o rekurencji.

Twierdzenie 1.1 (CLRS – Uniwersalne Twierdzenie o Rekurencji).

Niech $a \geq 1$, $b > 1$ będą stałymi, $f(n)$ funkcją i $T(n) = a \cdot T\left(\frac{n}{b}\right) + f(n)$ (gdzie $\frac{n}{b}$ znaczy $\left\lfloor \frac{n}{b} \right\rfloor$ lub $\left\lceil \frac{n}{b} \right\rceil$). Wtedy

$$T(n) = \begin{cases} \Theta(n^{\log_b a}) & \text{gdy } f(n) = \mathcal{O}(n^{\log_b a - \epsilon}) \text{ dla pewnego } \epsilon > 0 \\ \Theta(n^{\log_b a} \log n) & \text{gdy } f(n) = \Theta(n^{\log_b a}) \\ \Theta(f(n)) & \text{gdy } \begin{matrix} f(n) = \Omega(n^{\log_b a + \epsilon}) \text{ dla pewnego } \epsilon > 0 \\ a \cdot f(n/b) \leq c \cdot f(n) \text{ dla pewnej stałej } c < 1 \text{ dla dużych } n \end{matrix} \end{cases}$$

W praktyce, gdy człon nierekurencyjny jest wielomianowy ($f(n) = \Theta(n^k)$), wystarczy porównać b^k z a :

Wniosek 1.2. Niech $a \geq 1$, $b > 1$ będą stałymi, a $f(n) = \Theta(n^k)$ oraz $T(n) = a \cdot T\left(\frac{n}{b}\right) + f(n)$. Wtedy

$$T(n) = \begin{cases} \Theta(n^{\log_b a}) & \text{gdy } b^k < a \\ \Theta(n^k \log n) & \text{gdy } b^k = a \\ \Theta(n^k) & \text{gdy } b^k > a \end{cases}$$

Przykład 1.3 (Mnożenie macierzy).

A , B - macierze $n \times n$

$$A \cdot B = C = ?$$

Najpopularniejszy sposób mnożenia dwóch macierzy $n \times n$ działa w czasie $\mathcal{O}(n^3)$. Musimy obliczyć n^2 wartości i obliczenie każdej zajmuje liniowy czas.

$$\begin{matrix} B = & \begin{bmatrix} \vdots \end{bmatrix} \\ A = & \begin{bmatrix} \cdots \end{bmatrix} \end{matrix} \quad \begin{bmatrix} \vdots \\ \cdots \end{bmatrix} \times \begin{bmatrix} \vdots \\ \cdots \end{bmatrix} = C$$

Znajdywanie pary najbliższych punktów

Q - zbiór $n \geq 2$ punktów na płaszczyźnie

$$p_1 = (x_1, y_1) \quad p_2 = (x_2, y_2) \quad d(p_1, p_2) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

Problem: znaleźć parę punktów w najmniejszej odległości. Algorytm siłowy (*brute force*), który sprawdza wszystkie pary punktów działa w czasie $\Theta(n^2)$, bo mamy $\binom{n}{2} = \frac{n(n-1)}{2} = \Theta(n^2)$ par punktów. Pokażemy algorytm typu *dziel i zwyciężaj*, który działa w czasie $\mathcal{O}(n \log n)$.

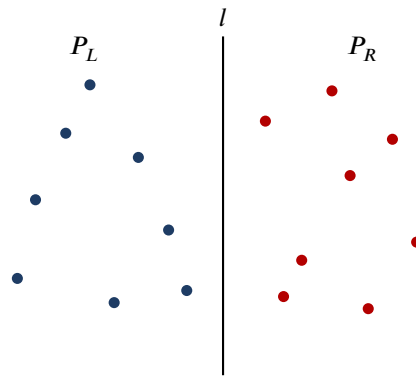
W każdym wywołaniu rekurencyjnym wejście składa się z 3 rzeczy: zbioru $P \subset Q$ i dwóch tablic X i Y które zawierają wszystkie punkty z P . W tablicy X punkty są posortowane w porządku niemalejącym względem pierwszej współrzędnej, a w tablicy Y po drugiej współrzędnej.

Opis algorytmu:

jeśli $|P| \leq 3$ sprawdzamy wszystkie pary

jeśli $|P| > 3$

Dziel: Dzielimy punkty P pionową prostą l na dwa podzbiory P_L i P_R na lewo i na prawo od l tak, że $|P_L| = \left\lceil \frac{|P|}{2} \right\rceil$, $|P_R| = \left\lfloor \frac{|P|}{2} \right\rfloor$. Dzielimy tablicę X na X_L i X_R zawierające punkty odpowiednio P_L i P_R posortowane w porządku niemalejącym względem pierwszej współrzędnej. Podobnie dzielimy Y na Y_L i Y_R zawierające odpowiednio punkty z P_L i P_R posortowane w porządku niemalejącym po drugiej współrzędnej.



Rysunek 1: Podział zbioru P pionową prostą l na podzbiory P_L (na lewo) i P_R (na prawo) o rozmiarach $|P_L| = \left\lceil \frac{|P|}{2} \right\rceil$ oraz $|P_R| = \left\lfloor \frac{|P|}{2} \right\rfloor$.

Zwycięzają: Wywołujemy rekurencyjnie algorytm dla wejścia P_L , X_L , Y_L i P_R , X_R , Y_R . Algorytm znajduje pary najbliższych punktów w P_L i P_R . Niech δ_L i δ_R będą najmniejszymi odległościami znalezionymi przez algorytm dla odpowiednio P_L i P_R . Niech δ będzie mniejszą z nich: $\delta = \min(\delta_L, \delta_R)$.

Połącz: Para najbliższych punktów z P składa się z dwóch punktów z P_L lub z dwóch punktów z P_R lub jednego punkty z P_L i jednego z P_R .

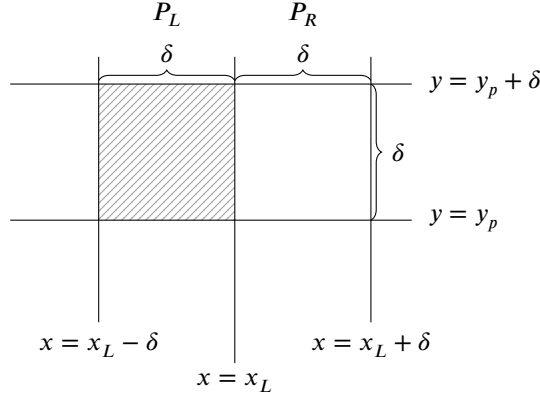
Musimy sprawdzić czy istnieje para 3-go typu w odległości mniejszej niż δ . Jeśli taka para istnieje, wtedy jej punkty są w pionowym pasie o szerokości 2δ wokół l .

W celu sprawdzenia czy taka para istnieje wykonujemy następujące czynności:

1. Tworzymy tablicę Y' otrzymaną z Y poprzez usunięcie punktów spoza pasa. Y' jest posortowany w porządku niemalejącym względem drugiej współrzędnej.
2. Dla każdego punktu p z Y' sprawdzamy odległość między p , a p' dla 7 kolejnych punktów p' z Y' . Algorytm oblicza najmniejszą odległość δ' i zapamiętuje ją i odpowiednią parę punktów.
3. Jeśli $\delta' < \delta$ to pionowy pas zawiera parę punktów w mniejszej odległości niż pary znalezione w wywołaniach rekurencyjnych. Algorytm zwraca δ' i odpowiednią parę punktów. Wpp. zwraca δ i odpowiednią parę punktów.

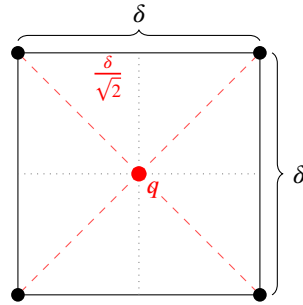
Dowód poprawności. Jedyną rzeczą, która nie jest oczywista jest fakt, że sprawdzamy tylko 7 następnych punktów po p w Y' w fazie połącz. Przypuśćmy, że p, q są parą punktów w najmniej-

szej odległości w P i $\delta(p, q) = \delta'$. Niech $p = (x_p, y_p)$, $q = (x_q, y_q)$. Bez straty ogólności załóżmy $y_p \leq y_q$. Pokażemy, że algorytm znajdzie tę parę punktów. Przypuśćmy, że nie. Wtedy istnieje co najmniej 7 punktów pomiędzy p i q w Y' . Niech $r = (x_r, y_r)$ będzie jednym z nich. Skoro $d(p, q) < \delta$ to $y_q - y_p = |y_q - y_p| \leq \sqrt{(x_p - x_q)^2 + (y_p - y_q)^2} = \delta(p, q) \leq \delta$. Ponadto y_r musi być między y_p i y_q : $y_p \leq y_r \leq y_q$, więc $y_r - y_p \leq y_q - y_p < \delta$ zatem co najmniej 9 punktów z P w prostokącie $2\delta \times \delta$ ograniczonym poziomymi liniami $y = y_p$, $y = y_p + \delta$.



Rysunek 2: Prostokąt $2\delta \times \delta$ ograniczony prostymi $y = y_p$ oraz $y = y_p + \delta$. Zacięniowany lewy kwadrat $\delta \times \delta$ zawiera co najmniej 5 punktów z P .

Wtedy w lewym lub prawym kwadracie $\delta \times \delta$ jest przynajmniej 5 punktów z P . Przypuśćmy, że to lewy kwadrat. Zatem istnieje kwadrat $\delta \times \delta$ z 5 punktami z P w P_L . Ale wtedy istnieją dwa punkty w P_L w odległości mniejszej niż δ , bo nie da się umieścić w kwadracie $\delta \times \delta$ 5 punktów dla których odległość między dowolnymi dwoma jest mniejsza niż δ . Sprzeczność (bo najmniejsza odległość w P_L to $\delta_L \geq \delta$).



Rysunek 3: W kwadracie $\delta \times \delta$ nie zmieści się 5 punktów parami oddalonych o co najmniej δ . Najlepsze ułożenie to 4 narożniki; dowolny piąty punkt q jest w odległości co najwyżej $\frac{\delta}{\sqrt{2}} < \delta$ od któregoś z nich (dwa z 5 punktów trafiają do tej samej ćwiartki $\frac{\delta}{2} \times \frac{\delta}{2}$).

Zatem wystarczy sprawdzić tylko 7 następnych punktów po p w Y' . □

Analiza złożoności czasowej Na początku musimy posortować tablicę X i Y . To zajmuje $\mathcal{O}(n \log n)$. Jeśli X i Y są już posortowane to podział X na posortowane X_L i X_R oraz Y analogicznie zajmie $\mathcal{O}(n)$.

Algorytm 1: Podział posortowanej tablicy Y na Y_L i Y_R

```

1:  $length(Y_L) \leftarrow length(Y_R) \leftarrow 0$ 
2: for  $i \leftarrow 1$  to  $length(Y)$  do
3:   if  $Y(i) \in P_L$  then
4:      $length(Y_L) \leftarrow length(Y_L) + 1$ 
5:      $Y_L(length(Y_L)) \leftarrow Y(i)$ 
6:   else
7:      $length(Y_R) \leftarrow length(Y_R) + 1$ 
8:      $Y_R(length(Y_R)) \leftarrow Y(i)$ 

```

Mamy dwa wywołania rekurencyjne dla $n/2$ punktów i fazę łącz. Tablicę Y' można otrzymać z Y w czasie $\mathcal{O}(n)$ tak, że Y' jest posortowana. Niech $x = x_L$ będzie równanie prostej l .

Algorytm 2: Wyznaczanie tablicy Y' punktów w pasie 2δ wokół prostej l

```

1:  $length(Y') \leftarrow 0$ 
2: for  $i \leftarrow 1$  to  $length(Y)$  do
3:   if  $x_L - \delta \leq Y(i).x \leq x_L + \delta$  then
4:      $length(Y') \leftarrow length(Y') + 1$ 
5:      $Y'(length(Y')) \leftarrow Y(i)$ 

```

Skoro dla każdego z $\mathcal{O}(n)$ punktów z Y' sprawdzamy odległość tylko co najwyżej 7 punktów, to najmniejsza odległość δ' punktów z Y' znajdziemy w czasie $\mathcal{O}(n)$.

Oznaczmy złożoność algorytmu bez wstępnego sortowania przez $T(n)$. Mamy:

$$T(n) = 2 \cdot T\left(\frac{n}{2}\right) + \Theta(n^1) \qquad a = 2, \quad b = 2, \quad k = 1$$

$$2 = 2^1 \Rightarrow T(n) = \Theta(n \log n)$$

Programowanie dynamiczne

Naiwna implementacja rekurencyjna bywa wykładnicza, bo te same podproblemy liczone są wielokrotnie (np. $FIB(n)$ przelicza $F_{n-1}, F_{n-2}, \dots$ od zera). Programowanie dynamiczne rozwiązuje każdy podproblem raz i zapamiętuje wynik w tablicy, schodząc z czasu wykładniczego do wielomianowego. Stosuje się je, gdy problem ma *optymalną podstrukturę* (rozwiązanie optymalne składa się z optymalnych rozwiązań podproblemów) i *nakładające się podproblemy*.

Algorytm mnożenia macierzy

Przypomnijmy, jak mnoży się macierze. Iloczyn $C = A \cdot B$ macierzy A rozmiaru $p \times q$ oraz B rozmiaru $q \times r$ jest macierzą C rozmiaru $p \times r$, w której element $C(i, j)$ to iloczyn skalarny i -tego wiersza macierzy A oraz j -tej kolumny macierzy B :

$$C(i, j) = \sum_{k=1}^q A(i, k) \cdot B(k, j).$$

Biorąc dla przykładu $p = 2, q = 3, r = 2$ i obliczając wyróżniony element $C(1, 1)$:

$$\underbrace{\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}}_{A: p \times q} \cdot \underbrace{\begin{bmatrix} 7 & 8 \\ 9 & 10 \\ 11 & 12 \end{bmatrix}}_{B: q \times r} = \underbrace{\begin{bmatrix} 58 & 64 \\ 139 & 154 \end{bmatrix}}_{C: p \times r}$$

$$C(1, 1) = 1 \cdot 7 + 2 \cdot 9 + 3 \cdot 11 = 7 + 18 + 33 = 58.$$

Kolorowy wiersz macierzy A i kolorowa kolumna macierzy B dają kolorowy element macierzy C ; sumowanie po $k = 1, \dots, q$ to wewnętrzna pętla algorytmu. Każdy z $p \cdot r$ elementów macierzy C wymaga q mnożeń.

Algorytm 3: Naiwne mnożenie macierzy

```

function MATRIXMULTIPLY( $A, B$ )  ▷  $A$  – macierz  $p \times q$ ;  $B$  – macierz  $q \times r$ ;  $C$  – macierz  $p \times r$ 
1:   for  $i \leftarrow 1$  to  $p$  do
2:     for  $j \leftarrow 1$  to  $r$  do
3:        $C(i, j) \leftarrow 0$ 
4:       for  $k \leftarrow 1$  to  $q$  do
5:          $C(i, j) \leftarrow C(i, j) + A(i, k) \cdot B(k, j)$   ▷ operacja dominująca
6:   return  $C$ 

```

Operacją dominującą jest mnożenie w linii 5. Mamy $p \cdot q \cdot r$ mnożeń. Złożoność jest $\mathcal{O}(p \cdot q \cdot r)$.

Problem mnożenia ciągu macierzy

Przykład 2.1 (Mnożenie ciągu macierzy).

(A_1, A_2, A_3) gdzie A_1 – macierz 5×10 , A_2 – macierz 10×3 , A_3 – macierz 3×2 . Ile potrzeba mnożeń skalarnych (mnożenia liczb) do obliczenia iloczynu $A_1 \cdot A_2 \cdot A_3$?

$$A_1 \cdot A_2 \cdot A_3 = (A_1 \cdot A_2) \cdot A_3 = A_1 \cdot (A_2 \cdot A_3)$$

I sposób: $(A_1 \cdot A_2) \cdot A_3 = 5 \cdot 10 \cdot 3 + 5 \cdot 3 \cdot 2 = 150 + 30 = 180$

II sposób: $A_1 \cdot (A_2 \cdot A_3) = 10 \cdot 3 \cdot 2 + 5 \cdot 10 \cdot 2 = 60 + 100 = 160$

II sposób jest szybszy.

Problem 2.2 (mnożenia ciągu macierzy).

Dla danego ciągu macierzy $(A_1, A_2, \dots, A_n)$, gdzie A_i jest macierzą $p_{i-1} \times p_i$ (cały ciąg wymiarów zapisujemy jednym ciągiem $p = (p_0, p_1, \dots, p_n)$ – kolejne macierze dzielą wspólny wymiar, więc n macierzy opisuje $n + 1$ liczb), znajdź kolejność wykonywania mnożeń minimalizującą liczbę mnożeń skalarnych użytych do obliczenia iloczynu $A_1 \cdot A_2 \cdot \dots \cdot A_n$.

Ile jest różnych sposobów obliczenia tego iloczynu? Innymi słowy, na ile sposobów możemy poprawnie wstawić nawiasy w iloczynie $A_1 \cdot A_2 \cdot \dots \cdot A_n$?

Przykład 2.3 (Kolejności nawiasowania dla $n = 4$).

$$A_1 \cdot A_2 \cdot A_3 \cdot A_4 \quad (n = 4)$$

- $((A_1 \cdot A_2) \cdot A_3) \cdot A_4$
- $(A_1 \cdot (A_2 \cdot A_3)) \cdot A_4$
- $A_1 \cdot ((A_2 \cdot A_3) \cdot A_4)$
- $A_1 \cdot (A_2 \cdot (A_3 \cdot A_4))$
- $(A_1 \cdot A_2) \cdot (A_3 \cdot A_4)$

Zatem mamy 5 sposobów.

Sprawdźmy czy umiemy rozwiązać ten problem szybko poprzez sprawdzenie wszystkich możliwości. Niech $P(n)$ – liczba sposobów wstawienia nawiasów w iloczynie n macierzy. Załóżmy, że ostatnie mnożenie w ciągu $A_1 \dots A_n$ to mnożenie między A_k i A_{k+1} : $(A_1 \dots A_k)(A_{k+1} \dots A_n)$. W podciągach lewym i prawym nawiasy są rozmieszczane niezależnie od siebie.

Stąd rekurencja:

$$\begin{cases} P(n) = \sum_{k=1}^{n-1} P(k) \cdot P(n-k) \\ P(1) = 1 \end{cases}$$

Jest to znana formuła rekurencyjna a rozwiązanie to przesunięta liczba Catalana:

$$P(n) = C(n-1), \text{ gdzie } C(n) = \frac{1}{n+1} \binom{2n}{n} \text{ jest } n\text{-tą liczbą Catalana.}$$

Wiadomo, że $C(n) = \Omega\left(\frac{4^n}{n^{3/2}}\right)$. Zatem $P(n)$ jest wykładniczy względem n . Chcemy znaleźć szybki algorytm.

Rozwiązanie z optymalną podstrukturą: Zauważmy że jeśli optymalne rozwiązanie jest postaci $(A_1 \dots A_k)(A_{k+1} \dots A_n)$ dla jakiegoś $k \in \{1, \dots, n-1\}$, to w podciągach $A_1 \dots A_k$ i $A_{k+1} \dots A_n$ rozwiązanie nawiasów musi być optymalne. Wpp. lepsze rozmieszczenie nawiasów w $A_1 \dots A_k$ lub $A_{k+1} \dots A_n$ dałoby lepsze rozmieszczenie nawiasów w całym ciągu.

Rozwiązanie optymalne zawiera rozwiązania optymalne podproblemów (własność optymalnej podstruktury). Podproblemami w naszym problemie są problemy optymalnego rozmieszczenia nawiasów w ciągach $A_i A_{i+1} \dots A_j$ dla $1 \leq i \leq j \leq n$.

Niech $m(i, j)$ oznacza minimalną liczbę mnożeń skalarnych potrzebnych do obliczenia tego iloczynu $A_i \dots A_j$ dla $1 \leq i \leq j \leq n$. Chcemy obliczyć $m(1, n)$.

$$m(i, i) = 0$$

Optymalny sposób rozmieszczenia nawiasów jest postaci $(A_i \dots A_k)(A_{k+1} \dots A_j)$ dla pewnego $k \in \{i, \dots, j-1\}$. Stąd mamy zależność:

$$m(i, j) = \min_{i \leq k < j} \{m(i, k) + m(k+1, j) + p_{i-1} \cdot p_k \cdot p_j\} \quad \text{dla } i < j$$

Niech $s(i, j)$, dla $1 \leq i < j \leq n$, będzie wartością k , dla którego to minimum jest osiągane (czyli $s(i, j)$ wskazuje miejsce ostatniego mnożenia w optymalnym rozwiązaniu podproblemu $A_i \dots A_j$).

Jeśli zaimplementujemy po prostu tę formułę rekurencyjną otrzymamy alg. wykładniczy. Ale jest tylko $n + \binom{n}{2} = \mathcal{O}(n^2)$ podproblemów, więc rzędu $\mathcal{O}(n^2)$ liczb $m(i, j)$ i każdą z tych liczb wystarczy obliczyć jeden raz.

Pomysł programowania dynamicznego polega więc na zapamiętaniu tych wartości w tablicy zamiast liczenia ich wielokrotnie. Wprowadzamy dwie tablice indeksowane parą (i, j) , $1 \leq i \leq j \leq n$ (a ponieważ rozważamy tylko $i \leq j$, wypełniona zostaje jedynie górna połowa każdej z nich – jak w przykładzie poniżej):

- $m(i, j)$ – **koszt**: minimalna liczba mnożeń skalarnych dla podproblemu $A_i \dots A_j$. Ostateczną odpowiedzią jest $m(1, n)$.

- $s(i, j)$ – **cięcie**: miejsce k ostatniego mnożenia, na którym ten minimalny koszt jest osiągnięty. Sama tablica m podaje tylko *ile* mnożeń wystarczy, ale nie *jak* ustawić nawiasy; to s pozwala odtworzyć optymalne nawiasowanie (wykorzysta ją algorytm MATRIXCHAINMULTIPLY dalej).

Algorytm 4: Algorytm Matrix Chain Order

```

function MATRIXCHAINORDER( $p$ )
1:    $n \leftarrow \text{length}(p) - 1$ 
2:   for  $i \leftarrow 1$  to  $n$  do
3:      $m(i, i) \leftarrow 0$ 
4:     for  $l \leftarrow 2$  to  $n$  do
5:       for  $i \leftarrow 1$  to  $n - l + 1$  do
6:          $j \leftarrow i + l - 1$ 
7:          $m(i, j) \leftarrow \infty$ 
8:         for  $k \leftarrow i$  to  $j - 1$  do
9:            $q \leftarrow m(i, k) + m(k + 1, j) + p_{i-1} \cdot p_k \cdot p_j$ 
10:          if  $q < m(i, j)$  then
11:             $m(i, j) \leftarrow q$ 
12:             $s(i, j) \leftarrow k$ 
13:   return  $m, s$ 

```

$\triangleright p = (p_0, p_1, \dots, p_n)$ - ciąg rozmiarów
 $\triangleright A_i$ - macierz $p_{i-1} \times p_i$
 $\triangleright l$ - liczba macierzy w podproblemie
 $\triangleright l = 2 : A_1 A_2, A_2 A_3, \dots, A_{n-1} A_n$
 $\triangleright l = 3 : A_1 A_2 A_3, A_2 A_3 A_4, \dots, A_{n-2} A_{n-1} A_n$
 $\triangleright l = n : A_1 \dots A_n$

Mamy 3 pętle (po l, i, k) i liczbę iteracji każdej z tych pętli można ograniczyć przez n , zatem złożoność czasowa to $\mathcal{O}(n^3)$.

Przykład 2.4 (Optymalna kolejność mnożenia macierzy dla $n = 5$).

$p = (4, 5, 2, 1, 2, 3)$.

$A_1 - 4 \times 5$, $A_2 - 5 \times 2$, $A_3 - 2 \times 1$, $A_4 - 1 \times 2$, $A_5 - 2 \times 3$.

$$m = \begin{bmatrix} i \backslash j & 1 & 2 & 3 & 4 & 5 \\ 1 & 0 & 40 & 30 & 38 & 48 \\ 2 & \times & 0 & 10 & 20 & 31 \\ 3 & \times & \times & 0 & 4 & 12 \\ 4 & \times & \times & \times & 0 & 6 \\ 5 & \times & \times & \times & \times & 0 \end{bmatrix}$$

$$s = \begin{bmatrix} i \backslash j & 1 & 2 & 3 & 4 & 5 \\ 1 & \times & 1 & 1 & 3 & 3 \\ 2 & \times & \times & 2 & 3 & 3 \\ 3 & \times & \times & \times & 3 & 3 \\ 4 & \times & \times & \times & \times & 4 \\ 5 & \times & \times & \times & \times & \times \end{bmatrix}$$

Prześledźmy, jak rosną obie tablice. Algorytm wypełnia je *przekątnymi*: najpierw $L = 1$ (przekątna główna), potem $L = 2$, $L = 3$, Komórki dopisane w danym kroku wyróżniono na **pomarańczowo**; puste pola to podproblemy jeszcze nieobliczone, a $\times$ – pola nieużywane ($i \geq j$ dla s , $i > j$ dla m).

Stan początkowy ($L = 1$, pojedyncze macierze, koszt 0):

$$m = \begin{bmatrix} i \backslash j & 1 & 2 & 3 & 4 & 5 \\ 1 & 0 & & & & \\ 2 & \times & 0 & & & \\ 3 & \times & \times & 0 & & \\ 4 & \times & \times & \times & 0 & \\ 5 & \times & \times & \times & \times & 0 \end{bmatrix}$$

$$s = \begin{bmatrix} i \backslash j & 1 & 2 & 3 & 4 & 5 \\ 1 & \times & & & & \\ 2 & \times & \times & & & \\ 3 & \times & \times & \times & & \\ 4 & \times & \times & \times & \times & \\ 5 & \times & \times & \times & \times & \times \end{bmatrix}$$

i	0	1	2	3	4	5
p_i	4	5	2	1	2	3

 $A_i = p_{i-1} \times p_i$

$$\begin{aligned}
 L = 2 : \quad m(1,2) &= 4 \cdot 5 \cdot 2 = 40 \\
 m(2,3) &= 5 \cdot 2 \cdot 1 = 10 \\
 m(3,4) &= 2 \cdot 1 \cdot 2 = 4 \\
 m(4,5) &= 1 \cdot 2 \cdot 3 = 6
 \end{aligned}$$

po $L = 2$ (pary sąsiednich macierzy):

$$m \begin{bmatrix} i \backslash j & 1 & 2 & 3 & 4 & 5 \\ 1 & 0 & 40 & & & \\ 2 & \times & 0 & 10 & & \\ 3 & \times & \times & 0 & 4 & \\ 4 & \times & \times & \times & 0 & 6 \\ 5 & \times & \times & \times & \times & 0 \end{bmatrix}$$

$$s \begin{bmatrix} i \backslash j & 1 & 2 & 3 & 4 & 5 \\ 1 & \times & 1 & & & \\ 2 & \times & \times & 2 & & \\ 3 & \times & \times & \times & 3 & \\ 4 & \times & \times & \times & \times & 4 \\ 5 & \times & \times & \times & \times & \times \end{bmatrix}$$

i	0	1	2	3	4	5
p_i	4	5	2	1	2	3

 $A_i = p_{i-1} \times p_i$

$$\begin{aligned}
 L = 3 : \quad m(1,3) &= \min \begin{cases} m(1,1) + m(2,3) + 4 \cdot 5 \cdot 1 \\ m(1,2) + m(3,3) + 4 \cdot 2 \cdot 1 \end{cases} = \min \begin{cases} 0 + 10 + 20 \\ 40 + 0 + 8 \end{cases} = \min \begin{cases} 30 \\ 48 \end{cases} = 30 \\
 m(2,4) &= \min \begin{cases} m(2,2) + m(3,4) + 5 \cdot 2 \cdot 2 \\ m(2,3) + m(4,4) + 5 \cdot 1 \cdot 2 \end{cases} = \min \begin{cases} 0 + 4 + 20 \\ 10 + 0 + 10 \end{cases} = \min \begin{cases} 24 \\ 20 \end{cases} = 20 \\
 m(3,5) &= \min \begin{cases} m(3,3) + m(4,5) + 2 \cdot 1 \cdot 3 \\ m(3,4) + m(5,5) + 2 \cdot 2 \cdot 3 \end{cases} = \min \begin{cases} 0 + 6 + 6 \\ 4 + 0 + 12 \end{cases} = \min \begin{cases} 12 \\ 16 \end{cases} = 12
 \end{aligned}$$

po $L = 3$:

$$m \begin{bmatrix} i \backslash j & 1 & 2 & 3 & 4 & 5 \\ 1 & 0 & 40 & 30 & & \\ 2 & \times & 0 & 10 & 20 & \\ 3 & \times & \times & 0 & 4 & 12 \\ 4 & \times & \times & \times & 0 & 6 \\ 5 & \times & \times & \times & \times & 0 \end{bmatrix}$$

$$s \begin{bmatrix} i \backslash j & 1 & 2 & 3 & 4 & 5 \\ 1 & \times & 1 & 1 & & \\ 2 & \times & \times & 2 & 3 & \\ 3 & \times & \times & \times & 3 & 3 \\ 4 & \times & \times & \times & \times & 4 \\ 5 & \times & \times & \times & \times & \times \end{bmatrix}$$

i	0	1	2	3	4	5
p_i	4	5	2	1	2	3

 $A_i = p_{i-1} \times p_i$

$$\begin{aligned}
 L = 4 : \quad m(1,4) &= \min \begin{cases} m(1,1) + m(2,4) + 4 \cdot 5 \cdot 2 \\ m(1,2) + m(3,4) + 4 \cdot 2 \cdot 2 \\ m(1,3) + m(4,4) + 4 \cdot 1 \cdot 2 \end{cases} = \min \begin{cases} 0 + 20 + 40 \\ 40 + 4 + 16 \\ 30 + 0 + 8 \end{cases} = \min \begin{cases} 60 \\ 60 \\ 38 \end{cases} = 38 \\
 m(2,5) &= \min \begin{cases} m(2,2) + m(3,5) + 5 \cdot 2 \cdot 3 \\ m(2,3) + m(4,5) + 5 \cdot 1 \cdot 3 \\ m(2,4) + m(5,5) + 5 \cdot 2 \cdot 3 \end{cases} = \min \begin{cases} 0 + 12 + 30 \\ 10 + 6 + 15 \\ 20 + 0 + 30 \end{cases} = \min \begin{cases} 42 \\ 31 \\ 50 \end{cases} = 31
 \end{aligned}$$

po $L = 4$:

m	$i \backslash j$	1	2	3	4	5
	1	0	40	30	38	
	2	×	0	10	20	31
	3	×	×	0	4	12
	4	×	×	×	0	6
	5	×	×	×	×	0

s	$i \backslash j$	1	2	3	4	5
	1	×	1	1	3	
	2	×	×	2	3	3
	3	×	×	×	3	3
	4	×	×	×	×	4
	5	×	×	×	×	×

i	0	1	2	3	4	5
p_i	4	5	2	1	2	3

$A_i = p_{i-1} \times p_i$

$$L = 5 : \quad m(1, 5) = \min \begin{cases} m(1, 1) + m(2, 5) + 4 \cdot 5 \cdot 3 \\ m(1, 2) + m(3, 5) + 4 \cdot 2 \cdot 5 \\ m(1, 3) + m(4, 5) + 4 \cdot 1 \cdot 3 \\ m(1, 4) + m(5, 5) + 4 \cdot 2 \cdot 3 \end{cases} = \min \begin{cases} 0 + 31 + 60 \\ 40 + 12 + 24 \\ 30 + 6 + 12 \\ 38 + 0 + 24 \end{cases} = \min \begin{cases} 91 \\ 76 \\ 48 \\ 62 \end{cases} = 48$$

po $L = 5$ (cały ciąg – ostatnia komórka):

m	$i \backslash j$	1	2	3	4	5
	1	0	40	30	38	48
	2	×	0	10	20	31
	3	×	×	0	4	12
	4	×	×	×	0	6
	5	×	×	×	×	0

s	$i \backslash j$	1	2	3	4	5
	1	×	1	1	3	3
	2	×	×	2	3	3
	3	×	×	×	3	3
	4	×	×	×	×	4
	5	×	×	×	×	×

Każdy przebieg pętli po l dopisuje dokładnie jedną przekątną: w komórce $m(i, j)$ łąduje koszt, a w bliźniaczej $s(i, j)$ – zwycięskie cięcie k . Ostatnią obliczoną wartością jest $m(1, 5) = 48$ w prawym górnym rogu – szukany koszt całego ciągu.

Oto algorytm znajdowania optymalnego rozwiązania:

Algorytm 5: Algorytm Matrix Chain Multiply

```

function MATRIXCHAINMULTIPLY( $A, s, i, j$ )                                ▷  $A = (A_1, \dots, A_n)$ 
1:   if  $j = i$  then
2:     return  $A_i$ 
3:    $X \leftarrow$  MATRIXCHAINMULTIPLY( $A, s, i, s(i, j)$ )
4:    $Y \leftarrow$  MATRIXCHAINMULTIPLY( $A, s, s(i, j) + 1, j$ )
5:   return MATRIXMULTIPLY( $X, Y$ )

```

W celu znalezienia rozwiązania wywołujemy: MATRIXCHAINMULTIPLY($A, s, 1, n$)

$$A_1 \cdot A_2 \cdot A_3 \cdot A_4 \cdot A_5$$

$$s(1, 5) = 3 \implies (A_1 \cdot A_2 \cdot A_3)(A_4 \cdot A_5)$$

$$s(1, 3) = 1 \implies (A_1 \cdot (A_2 \cdot A_3))(A_4 \cdot A_5) \quad s(4, 5) = 4 \implies \text{bez zmian}$$

$$s(2, 3) = 2 \implies (A_1 \cdot (A_2 \cdot A_3))(A_4 \cdot A_5)$$

Aby algorytm zadziałał dobrze, problem musi spełniać warunki:

1. Własność optymalnej podstruktury – optymalne rozwiązanie problemu można skonstruować z optymalnych rozwiązań podproblemów.

2. Własność wspólnych podproblemów – sytuacja, gdy prosty algorytm rekurencyjny obliczałby rozwiązania tych samych podproblemów wielokrotnie. Liczba podproblemów powinna być „mała”.

W klasycznej metodzie programowania dynamicznego rozwiązanie jest budowane od dołu do góry – od podproblemów najmniejszych do największych. Jest też podejście, w którym rozwiązanie jest budowane od góry do dołu, podobnie jak w prostym algorytmie rekurencyjnym, ale z tą różnicą, że unikamy obliczania tych samych podproblemów wielokrotnie. To podejście nazywa się spamiętywaniem (*memoization*).

Problem mnożenia ciągu macierzy ze spamiętywaniem Algorytm oblicza tablice m i s jak poprzednio, ale w inny sposób. Dopóki podproblem nie jest rozwiązany, $m(i, j)$ przechowuje ∞ . Po rozwiązaniu podproblemu (i, j) umieszczamy w $m(i, j)$ odpowiednią wartość i od tej pory używamy jej bez obliczania.

Algorytm 6: Algorytm Lookup Chain

function LOOKUPCHAIN(p, i, j) $\triangleright p = (p_0, \dots, p_n)$ - ciąg wymiarów; A_i - macierz $p_{i-1} \times p_i$

```

1:  if  $m(i, j) < \infty$  then
2:    return  $m(i, j), s(i, j)$ 
3:  if  $i = j$  then
4:     $m(i, j) \leftarrow 0$ 
5:  else
6:    for  $k \leftarrow i$  to  $j - 1$  do
7:       $q \leftarrow \text{LOOKUPCHAIN}(p, i, k) + \text{LOOKUPCHAIN}(p, k + 1, j) + p_{i-1} \cdot p_k \cdot p_j$ 
8:      if  $q < m(i, j)$  then
9:         $m(i, j) \leftarrow q$ 
10:        $s(i, j) \leftarrow k$ 
11:  return  $(m(i, j), s(i, j))$ 

```

Algorytm 7: Algorytm Memoized Matrix Chain

function MEMOIZEDMATRIXCHAIN(p)

```

1:   $n \leftarrow \text{length}(p) - 1$ 
2:  for  $i \leftarrow 1$  to  $n$  do
3:    for  $j \leftarrow i$  to  $n$  do
4:       $m(i, j) \leftarrow \infty$ 
5:  return LOOKUPCHAIN( $p, 1, n$ )

```

Złożoność czasowa:

- Linie 1–4 w MEMOIZEDMATRIXCHAIN zajmują $\Theta(n^2)$.
- Każdy z $\mathcal{O}(n^2)$ elementów tablicy m jest modyfikowany raz, później odczytanie wartości $m(i, j)$ zajmuje czas stały.
- Modyfikacja $m(i, j)$ zajmuje $\mathcal{O}(n)$ czasu (pętla w liniach 5–9 w LOOKUPCHAIN).

Ostatecznie złożoność algorytmu wynosi $\mathcal{O}(n^3)$.

Problem najdłuższego wspólnego podciągu

Niech $X = (x_1, \dots, x_m)$ – ciąg.

$Z = (z_1, \dots, z_k)$ jest podciągiem X , jeśli istnieje rosnący ciąg indeksów $1 \leq i_1 < i_2 < \dots < i_k \leq m$ takich, że dla każdego $j \in \{1, \dots, k\}$, $z_j = x_{i_j}$.

Przykład 2.5 (Podciąg).

$X = (A, B, C, B, D, A, B)$, $Z = (B, C, D, B)$. Indeksy: (2, 3, 5, 7).

Problem 2.6 (najdłuższego wspólnego podciągu).

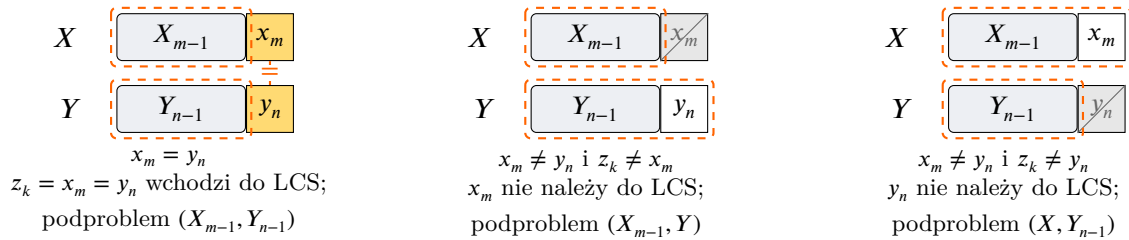
Dane są ciągi $X = (x_1, \dots, x_m)$ oraz $Y = (y_1, \dots, y_n)$. Należy znaleźć najdłuższy wspólny podciąg (LCS – *Longest Common Subsequence*) X i Y , czyli najdłuższy ciąg, który jest podciągiem zarówno X , jak i Y .

Wszystkich podciągów X jest 2^m , więc algorytm sprawdzający wszystkie podciągi X działa w czasie wykładniczym.

Niech $X_i = (x_1, \dots, x_i)$ dla $i \in \{1, \dots, m\}$ i dodatkowo $X_0 = ()$ (ciąg pusty). Analogicznie dla Y .

Lemat 2.7. Niech $X = (x_1, \dots, x_m)$, $Y = (y_1, \dots, y_n)$ i niech $Z = (z_1, \dots, z_k)$ będzie LCS X i Y .

1. Jeśli $x_m = y_n$, to $z_k = x_m = y_n$ i Z_{k-1} jest LCS X_{m-1} i Y_{n-1} .
2. Jeśli $x_m \neq y_n$ i $z_k \neq x_m$, to Z jest LCS X_{m-1} i Y .
3. Jeśli $x_m \neq y_n$ i $z_k \neq y_n$, to Z jest LCS X i Y_{n-1} .



Rysunek 4: Trzy przypadki lematu o optymalnej podstrukturze LCS. Żółta komórka – dopasowany ostatni element ($x_m = y_n$), który wchodzi do LCS; szara, przekreślona komórka – element odrzucony, nie należący do tego LCS; przerywana pomarańczowa ramka – podproblem(y), do których redukuje się zadanie.

Rozwiązanie dla problemu (X, Y) można skonstruować z rozwiązań podproblemów (X_{m-1}, Y_{n-1}) , (X_{m-1}, Y) i (X, Y_{n-1}) – **własność optymalnej podstruktury**. Podproblem (X_{m-1}, Y_{n-1}) jest zawarty w podproblemach (X_{m-1}, Y) i (X, Y_{n-1}) – **własność wspólnych podproblemów**.

Niech $c(i, j)$ oznacza długość LCS ciągów X_i i Y_j .

$$c(i, j) = \begin{cases} 0 & \text{jeśli } i = 0 \text{ lub } j = 0 \\ c(i-1, j-1) + 1 & \text{jeśli } i, j > 0 \text{ i } x_i = y_j \\ \max\{c(i-1, j), c(i, j-1)\} & \text{jeśli } i, j > 0 \text{ i } x_i \neq y_j \end{cases}$$

Będziemy też używać tablicy $b(i, j)$, aby znaleźć sam podciąg LCS. Tablice mają wymiary: $c[0 \dots m, 0 \dots n]$ oraz $b[1 \dots m, 1 \dots n]$. Wartości w tablicy kierunków to $b(i, j) \in \{\nwarrow, \uparrow, \leftarrow\}$.

Algorytm 8: Algorytm obliczający długość LCS

```
function LCS(X, Y)
1:  m ← length(X)
2:  n ← length(Y)
3:  for i ← 0 to m do
```

```

4:    $c(i, 0) \leftarrow 0$ 
5:   for  $j \leftarrow 0$  to  $n$  do
6:      $c(0, j) \leftarrow 0$ 
7:   for  $i \leftarrow 1$  to  $m$  do
8:     for  $j \leftarrow 1$  to  $n$  do
9:       if  $x_i = y_j$  then
10:         $c(i, j) \leftarrow c(i-1, j-1) + 1$ 
11:         $b(i, j) \leftarrow \text{„}\searrow\text{”}$ 
12:       else if  $c(i-1, j) \geq c(i, j-1)$  then
13:         $c(i, j) \leftarrow c(i-1, j)$ 
14:         $b(i, j) \leftarrow \text{„}\uparrow\text{”}$ 
15:       else
16:         $c(i, j) \leftarrow c(i, j-1)$ 
17:         $b(i, j) \leftarrow \text{„}\leftarrow\text{”}$ 
18:   return  $(c, b)$ 

```

Przykład: $X = (A, B, C, B, D, A, B)$, $Y = (B, D, C, A, B, A)$.
 $m = 7$, $n = 6$.

Tablica kosztów c :

	j	0	1	2	3	4	5	6
i	y_j	B	D	C	A	B	A	
0	x_i	0	0	0	0	0	0	0
1	A	0	0	0	0	1	1	1
2	B	0	1	1	1	1	2	2
3	C	0	1	1	2	2	2	2
4	B	0	1	1	2	2	3	3
5	D	0	1	2	2	2	3	3
6	A	0	1	2	2	3	3	4
7	B	0	1	2	2	3	4	4

Tablica strzałek b :

	j	0	1	2	3	4	5	6
i	y_j	B	D	C	A	B	A	
0	x_i							
1	A		$\uparrow$	$\uparrow$	$\uparrow$	$\searrow$	$\leftarrow$	$\searrow$
2	B		$\searrow$	$\leftarrow$	$\leftarrow$	$\uparrow$	$\searrow$	$\leftarrow$
3	C		$\uparrow$	$\uparrow$	$\searrow$	$\leftarrow$	$\uparrow$	$\uparrow$
4	B		$\searrow$	$\uparrow$	$\uparrow$	$\uparrow$	$\searrow$	$\leftarrow$
5	D		$\uparrow$	$\searrow$	$\uparrow$	$\uparrow$	$\uparrow$	$\uparrow$
6	A		$\uparrow$	$\uparrow$	$\uparrow$	$\searrow$	$\uparrow$	$\searrow$
7	B		$\searrow$	$\uparrow$	$\uparrow$	$\uparrow$	$\searrow$	$\uparrow$

Algorytm 9: Wypisywanie najdłuższego wspólnego podciągu

```

function PRINTLCS( $b, X, Y, i, j$ )
1:   if  $i = 0$  lub  $j = 0$  then
2:     return  $\varepsilon$ 
3:   if  $b(i, j) = \text{„}\searrow\text{”}$  then
4:     PRINTLCS( $b, X, Y, i-1, j-1$ )
5:     print  $x_i$ 
6:   else if  $b(i, j) = \text{„}\uparrow\text{”}$  then
7:     PRINTLCS( $b, X, Y, i-1, j$ )
8:   else
9:     PRINTLCS( $b, X, Y, i, j-1$ )

```

▷ ciąg pusty

Żeby wypisać rozwiązanie, wywołujemy PRINTLCS(b, X, Y, m, n).

Dla powyższego przykładu otrzymamy: (B, C, B, A) .

Złożoność czasowa tego algorytmu wynosi $\Theta(m \cdot n)$.

Algorytmy zachłanne

Pomysł: Algorytm w każdym kroku dokonuje lokalnie najlepszego wyboru (najlepszego w danym momencie).

Problem 3.1 (wyboru zajęć).

$S = \{z_1, \dots, z_n\}$ – zbiór zajęć wymagający tych samych zasobów.

s_i – czas rozpoczęcia zajęcia z_i

$s_i, t_i \in \mathbb{N}, \quad s_i < t_i$

t_i – czas zakończenia zajęcia z_i

Zajęcia z_i i z_j nazywamy zgodnymi, jeśli $\langle s_i, t_i \rangle \cap \langle s_j, t_j \rangle = \emptyset$. Zadanie polega na znalezieniu podzbioru o największej liczności $A \subseteq S$ parami zgodnych zajęć.

Przykład 3.2. Przykładowy zbiór zajęć:

i	1	2	3	4	5	6
s_i	0	2	4	7	1	7
t_i	4	6	7	10	3	9

Porządkujemy zajęcia w czasie niemalejącym względem czasu zakończenia. To zajmuje $\mathcal{O}(n \log n)$ czasu. Od tej pory zakładamy $t_1 \leq t_2 \leq \dots \leq t_n$ (zajęcia są już posortowane).

Przykład 3.3. Zajęcia z przykładu 3.2 po posortowaniu względem czasu zakończenia:

i	1	2	3	4	5	6
s_i	1	0	2	4	7	7
t_i	3	4	6	7	9	10

Problem ma optymalną podstrukturę: jeśli z_k należy do rozwiązania optymalnego, to po obu stronach z_k pozostają niezależne podproblemy, których rozwiązania też muszą być optymalne. Daje to algorytm programowania dynamicznego o złożoności $\mathcal{O}(n^3)$, ale – jak pokazuje poniższy lemat – wystarczy podejście zachłanne.

Algorytm zachłanny

Lemat 3.4. Niech $z_m \in S_{ij}$ będzie zajęciem takim, że $t_m = \min\{t_k : z_k \in S_{ij}\}$. Wtedy:

1. Istnieje podzbiór o największej liczności składający się z parami zgodnych zajęć zbioru S_{ij} zawierający z_m .
2. $S_{im} = \emptyset$ (zatem po wybraniu z_m do zbioru rozwiązania zostaje co najwyżej 1 niepusty podproblem do rozwiązania – S_{mj}).

Z lematu wynika, że nie musimy sprawdzać $j - i - 1$ wartości k , możemy wybrać zajęcie o najmniejszym czasie zakończenia $z_m \in S_{ij}$. Żeby rozwiązać podproblem S_{ij} możemy dodać z_m do zbioru rozwiązania a następnie dodać do zbioru rozwiązania rozwiązanie optymalne podproblemu S_{mj} . Zaczynamy z $S = S_{0,n+1}$ i $t_0 = 0$. W pierwszym kroku algorytm wybiera z_1 .

Algorytm 10: Zachłanny wybór zajęć

<pre> function GREEDYACTIVITYSELECTOR(s, t) 1: $n \leftarrow \text{length}(s)$ 2: $A \leftarrow \{z_1\}$ 3: $j \leftarrow 1$ 4: for $i \leftarrow 2$ to n do 5: if $s_i \geq t_j$ then 6: $A \leftarrow A \cup \{z_i\}$ 7: $j \leftarrow i$ 8: return A </pre>	$\triangleright$ zajęcia są posortowane po czasie zakończeń $\triangleright A$ - zbiór rozwiązań
---	--

Algorytm zachłanny w każdym kroku wybiera zajęcie zgodne ze wszystkimi zajęciami poprzednio wybranymi, o najmniejszym czasie zakończenia, aby zostawić jak najwięcej miejsca dla zajęć następnych (algorytm zachłanny). Skoro one są posortowane, jeśli $j < k$ i z_k jest zgodne z z_j , to z_k jest też zgodne z z_i dla każdego $1 \leq i < j$.

Analiza poprawności: Algorytm zwraca zbiór parami zgodnych zajęć, bo dodaje do zbioru rozwiązania zajęcie tylko jeśli jest zgodne ze wszystkimi zajęciami dodanymi wcześniej.

Pokażemy przez indukcję po n , że algorytm znajduje rozwiązanie optymalne.

- Dla $n = 1$ oczywiste ✓
- Niech $n > 1$. Z lematu istnieje optymalne rozwiązanie A_1 takie, że $z_1 \in A_1$.
Rozważmy problem dla $S' = \{z_i \in S : s_i \geq t_1\}$. Oczywiście zachodzi $|S'| < |S|$.
Dla każdego $z_j \in A_1 \setminus \{z_1\}$, mamy $z_j \in S'$, bo $s_j \geq t_1$, co wynika z faktu, że A_1 jest rozwiązaniem dla S .
Pokażemy, że $A_1 \setminus \{z_1\}$ jest rozwiązaniem optymalnym dla S' .
Przypuśćmy, że istnieje rozwiązanie B dla S' takie, że $|B| > |A_1 \setminus \{z_1\}|$. Wtedy $\{z_1\} \cup B$ byłoby rozwiązaniem dla S takim, że:

$$|\{z_1\} \cup B| = 1 + |B| > 1 + |A_1 \setminus \{z_1\}| = |A_1|$$

Otrzymujemy sprzeczność, bo A_1 jest z założenia rozwiązaniem optymalnym.
Z założenia indukcyjnego nasz algorytm znajduje rozwiązanie optymalne B' dla S' . Wtedy $|B'| = |A_1 \setminus \{z_1\}| = |A_1| - 1$.
Zbiór $\{z_1\} \cup B'$ jest rozwiązaniem znalezionym przez algorytm dla S i:

$$|\{z_1\} \cup B'| = 1 + |B'| = 1 + |A_1| - 1 = |A_1|$$

Więc $\{z_1\} \cup B'$ jest rozwiązaniem optymalnym.

Złożoność czasowa

- linia 1 – $\mathcal{O}(n)$
- linia 2 – $\mathcal{O}(1)$
- iteracji pętli w linii 3 jest $n - 1$
- wewnątrz pętli zajmuje czas $\mathcal{O}(1)$

Całość zajmuje $\Theta(n)$, jeśli zajęcia są już posortowane po czasie zakończenia. Ostatecznie złożoność algorytmu wynosi (uwzględniając początkowe sortowanie):

$$\mathcal{O}(n \log n) + \Theta(n) = \mathcal{O}(n \log n)$$

Kody Huffmana

Często znaki są reprezentowane przy użyciu 8 bitów (np. ASCII). Skoro częstość znaków się różni, lepiej reprezentować znaki występujące często przy użyciu krótszych ciągów bitów, a znaki występujące rzadko przy użyciu dłuższych ciągów bitów.

częstość = liczba wystąpień

Przykład W pliku jest 58 znaków. Każdy znak jest elementem zbioru $\{a, e, i, s, t, r, n\}$.

		<i>a</i>	<i>e</i>	<i>i</i>	<i>s</i>	<i>t</i>	<i>r</i>	<i>n</i>
	częstość	10	15	12	3	4	13	1
kod 1	stała długość słowa kodowego	000	001	010	011	100	101	110
kod 2	zmienna długość słowa kodowego	001	01	10	00000	0001	11	00001
kod 3	zmienna długość słowa kodowego	001	01	10	00010	0001	11	00001

Jeśli użyjemy kodu o stałej długości słowa kodowego (kod 1) to zakodowany plik ma: $58 \cdot 3 = 174$ bity.

Jeśli użyjemy kodu o zmiennej długości słowa kodowego (kod 2 lub kod 3) to zakodowany plik ma:

$$10 \cdot 3 + 15 \cdot 2 + 12 \cdot 2 + 3 \cdot 5 + 4 \cdot 4 + 13 \cdot 2 + 1 \cdot 5 = 146 \text{ bitów.}$$

Jeśli używamy kodu o zmiennej długości słowa kodowego, to chcemy aby spełniony był warunek:
żadne słowo kodowe nie jest prefixem innego słowa kodowego.

Takie kody nazywa się kodami prefixowymi (choć powinno być bezprefixowe).

Rozważmy kod 3. Ciąg 0001001 można odkodować na dwa sposoby:

- 00010 01 $\Rightarrow s \ e$
- 0001 001 $\Rightarrow t \ a$

Tutaj 0001 jest prefixem 00010, czyli słowo kodowe znaku *t* jest prefixem słowa kodowego znaku *s*.

Reprezentacja drzewowa kodów Słowo kodowe znaku jest ciągiem etykiet krawędzi na ścieżce od korzenia do wierzchołka zaetykietowanego tym znakiem.

Uwaga. Żadne słowo kodowe nie jest prefixem innego słowa kodowego $\iff$ wszystkie znaki są reprezentowane przez liście w reprezentacji drzewowej kodu.

Etykietujemy wierzchołki w następujący sposób: każdy wierzchołek v dostaje etykietę będącą sumą wystąpień znaków, które są reprezentowane przez liście poddrzewa ukorzonego w v .

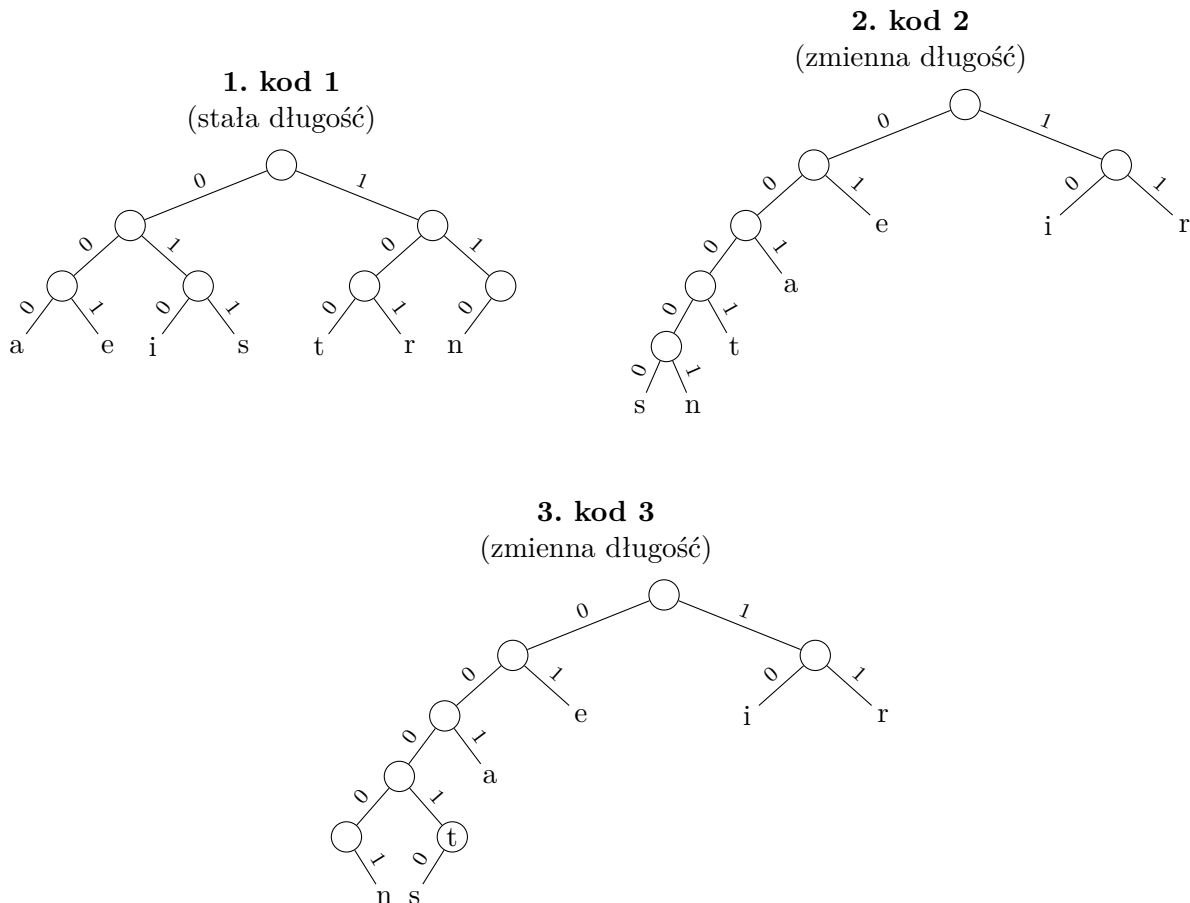
Niech C będzie alfabetem, $f : C \rightarrow \mathbb{N}_+$, $f(c) =$ częstość znaku $c \in C$.

$d_T(c)$ – głębokość liścia reprezentującego znak $c \in C$ w drzewie T

$d_T(c) =$ długość słowa kodowego znaku $c \in C$ w kodzie reprezentowanym przez drzewo T

Liczba bitów w zakodowanym pliku (kod reprezentowany przez drzewo T) wynosi:

$$B(T) = \sum_{c \in C} f(c) \cdot d_T(c)$$



Kod prefixowy reprezentowany przez drzewo T nazywamy optymalnym, jeśli $B(T)$ jest najmniejsze możliwe.

Drzewo ukorzenione nazywamy binarnym, jeśli każdy wierzchołek ma co najwyżej dwoje dzieci. Drzewo binarne nazywamy regularnym, jeśli każdy wierzchołek wewnętrzny ma dokładnie dwoje dzieci.

Twierdzenie 3.5. *Drzewo binarne reprezentujące optymalny kod jest regularne.*

Algorytm Huffmana

Algorytm konstruuje drzewo reprezentujące optymalny kod prefixowy dla (C, f) . Zaczynamy od $|C|$ wierzchołków izolowanych (które staną się liśćmi) i wykonujemy $|C| - 1$ operacji łączenia zaczynając od najrzadszych znaków (będą miały największą głębokość w drzewie).

Będziemy używać kolejki priorytetowej z kluczem $f(c)$. Kolejkę priorytetową możemy zaimplementować używając kopca binarnego. Wtedy koszty operacji wynoszą:

- INSERT – $\mathcal{O}(\log n)$
- EXTRACT_MIN – $\mathcal{O}(\log n)$
- CREATE – $\mathcal{O}(n)$

gdzie n jest liczbą elementów w kolejce.

Algorytm 11: Algorytm Huffmana

```

function HUFFMAN( $C, f$ )
1:   $n \leftarrow |C|$ 
2:   $Q \leftarrow \text{CREATE}(C, f)$ 
3:  for  $i \leftarrow 1$  to  $n - 1$  do
4:     $z \leftarrow \text{NEW\_NODE}$ 
5:     $x \leftarrow \text{LEFT}(z) \leftarrow \text{EXTRACT\_MIN}(Q)$ 
6:     $y \leftarrow \text{RIGHT}(z) \leftarrow \text{EXTRACT\_MIN}(Q)$ 
7:     $f(z) \leftarrow f(x) + f(y)$ 
8:     $\text{INSERT}(Q, z)$ 
9:  return  $\text{EXTRACT\_MIN}(Q)$ 
    
```

$\triangleright Q$ - kolejka priorytetowa

Algorytm konstruuje drzewo binarne. z - wierzchołek, $left, right$ - dzieci.

Złożoność czasowa

- linia 1 – $\mathcal{O}(1)$
- linia 2 – $\mathcal{O}(n)$
- wewnątrz pętli z linii 3 wykona się $n - 1$ razy:
 - linia 4 – $\mathcal{O}(1)$
 - linie 5, 6 – $\mathcal{O}(\log n)$
 - linia 7 – $\mathcal{O}(1)$
 - linia 8 – $\mathcal{O}(\log n)$

Wewnątrz pętli zajmuje łącznie czas $\mathcal{O}(\log n)$.

$$\mathcal{O}(n) + (n - 1) \cdot \mathcal{O}(\log n) = \mathcal{O}(n \log n)$$

Złożoność czasowa algorytmu wynosi $\mathcal{O}(n \log n)$.

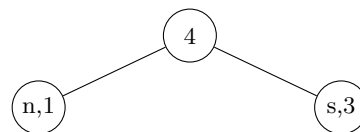
Przykład 3.6 (Konstrukcja drzewa Huffmana).

Zaczynamy ze zbiorem wierzchołków: $(n, 1), (s, 3), (t, 4), (a, 10), (i, 12), (r, 13), (e, 15)$.

1. Łączymy n i s .

Zostają:

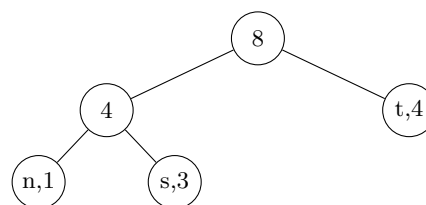
$(t, 4), (a, 10), (i, 12), (r, 13), (e, 15)$.



2. Łączymy 4 i t .

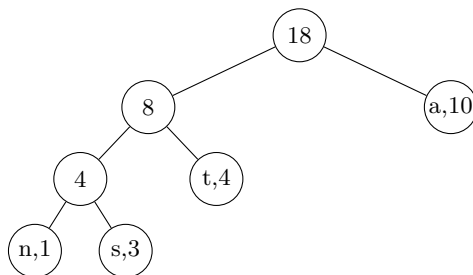
Zostają:

$(a, 10), (i, 12), (r, 13), (e, 15)$.



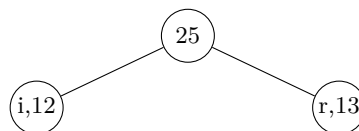
3. Łączymy 8 i a .

Zostają: $(i, 12), (r, 13), (e, 15)$.

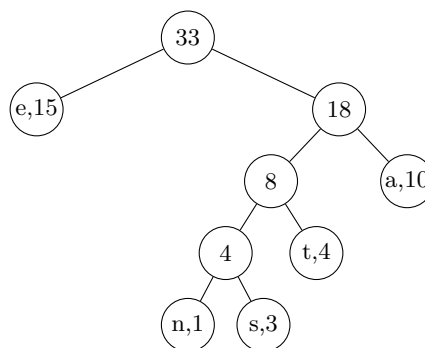


4. Łączymy i i r .

Zostaje: $(e, 15)$.

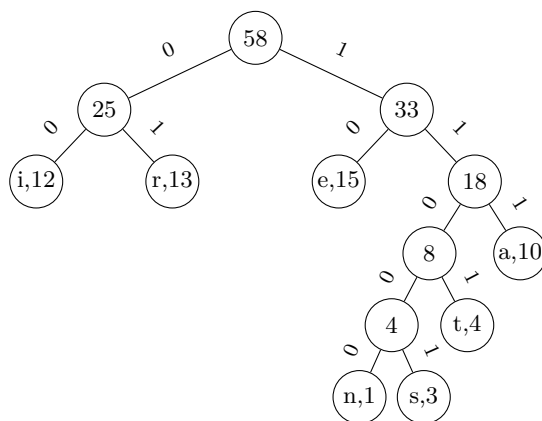


5. Łączymy e i 18.



6. Łączymy 25 i 33.

(ostateczne drzewo kodów)



Odczytany kod z drzewa w kroku 6:

$i \Rightarrow 00$
 $r \Rightarrow 01$
 $e \Rightarrow 10$
 $n \Rightarrow 11000$
 $s \Rightarrow 11001$
 $t \Rightarrow 1101$
 $a \Rightarrow 111$

Dowód optymalności algorytmu Huffmana

Kody prefiksowe $\iff$ regularne drzewa binarne.

C – alfabet, $f : C \rightarrow \mathbb{N}$ funkcja częstości, $d_T(c)$ – głębokość liścia reprezentującego c w drzewie T reprezentującym kod prefiksowy.

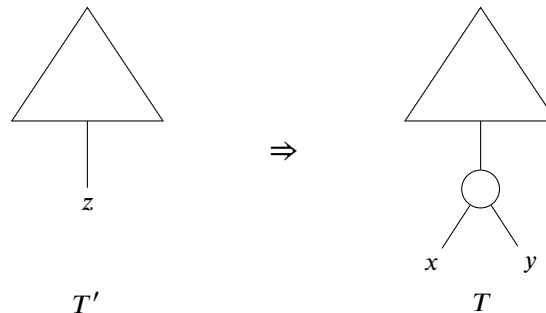
$$B(T) = \sum_{c \in C} f(c) \cdot d_T(c) \longleftarrow \min$$

Lemat 3.7. Niech x i y będą najrzadziej występującymi znakami z C . Istnieje optymalny kod prefiksowy, w którym słowa kodowe x i y mają tę samą długość i różnią się tylko na ostatnim bicie.

Z lematu 3.7 wynika, że można zacząć konstrukcję optymalnego drzewa połączeniem dwóch symboli o najmniejszej częstości. **Zachłanny wybór** – w każdym kroku algorytm łączy dwa wierzchołki o najmniejszych kluczach.

Lemat 3.8. Niech A będzie alfabetem z funkcją częstości $f : A \rightarrow \mathbb{N}$, a $x, y \in A$ – dwoma znakami o najmniejszych częstościach. Zdefiniujmy zredukowany alfabet $A' = (A \setminus \{x, y\}) \cup \{z\}$, gdzie $z \notin A$, z funkcją częstości f' taką, że $f'(c) = f(c)$ dla $c \neq z$ oraz $f'(z) = f(x) + f(y)$.

Jeśli T' jest optymalnym drzewem kodu prefiksowego dla (A', f') , to drzewo T powstałe z T' przez zastąpienie liścia z wewnętrznym węzłem z dziećmi x i y jest optymalnym drzewem kodu prefiksowego dla (A, f) .



Twierdzenie 3.9. Algorytm Huffmana konstruuje drzewo reprezentujące optymalny kod prefiksowy.

Szeregowanie zadań**Problem 3.10 (Szeregowanie zadań – jeden procesor).**

Jak uszeregować zadania tak, aby średni czas ukończenia był jak najmniejszy?

Przykład 3.11 (Szeregowanie zadań – jeden procesor).

i	zadanie	t_i
1	z_1	15
2	z_2	8
3	z_3	3
4	z_4	10

Rozważmy uszeregowanie z_1, z_2, z_3, z_4 :

$$\frac{15 + (15 + 8) + (15 + 8 + 3) + (15 + 8 + 3 + 10)}{4} = \frac{15 + 23 + 26 + 36}{4} = \frac{100}{4} = 25$$

Rozważmy uszeregowanie z_3, z_2, z_4, z_1 :

$$\frac{3 + (3 + 8) + (3 + 8 + 10) + (3 + 8 + 10 + 15)}{4} = \frac{3 + 11 + 21 + 36}{4} = 17\frac{3}{4}$$

Rozwiązanie: Uszereguj zadania od najkrótszego do najdłuższego.

Problem 3.12 (Szeregowanie zadań – 3 procesory).

Mamy 3 procesory. Pytanie jak poprzednio (por. 3.11).

Przykład 3.13 (Szeregowanie zadań – trzy procesory).

Zadanie	z_1	z_2	z_3	z_4	z_5	z_6	z_7	z_8	z_9
t_i	3	5	6	10	11	14	15	18	20

Rozwiązanie: Przydzielaj zadania do pierwszego wolnego procesora w kolejności od najkrótszego do najdłuższego.

	3	5	6	13	16	20	28	34	40
P_1	z_1		z_4			z_7			
P_2	z_2		z_5			z_8			
P_3	z_3		z_6				z_9		

W problemie 3.11 dla uszeregowania $z_{i_1}, z_{i_2}, \dots, z_{i_n}$ suma zakończeń zadań wynosi:

$$\begin{aligned} \sum_{j=1}^n \sum_{k=1}^j t_{i_k} &= t_{i_1} + (t_{i_1} + t_{i_2}) + (t_{i_1} + t_{i_2} + t_{i_3}) + \dots + (t_{i_1} + \dots + t_{i_n}) = \\ &= n \cdot t_{i_1} + (n-1)t_{i_2} + (n-2)t_{i_3} + \dots + 1 \cdot t_{i_n} = \sum_{j=1}^n (n+1-j)t_{i_j} \end{aligned}$$

W problemie 3.12 (dla $3 \mid n$) dla uszeregowania $z_{i_1}, z_{i_2}, \dots, z_{i_n}$ suma czasów zakończeń wynosi:

$$\begin{aligned} &\underbrace{t_{i_1} + t_{i_2} + t_{i_3}} + \underbrace{(t_{i_1} + t_{i_4}) + (t_{i_2} + t_{i_5}) + (t_{i_3} + t_{i_6})} + \underbrace{(t_{i_1} + t_{i_4} + t_{i_7}) + (t_{i_2} + t_{i_5} + t_{i_8}) + (t_{i_3} + t_{i_6} + t_{i_9})} + \\ &+ \dots + \underbrace{(t_{i_1} + t_{i_4} + \dots + t_{i_{n-2}}) + (t_{i_2} + t_{i_5} + \dots + t_{i_{n-1}}) + (t_{i_3} + t_{i_6} + \dots + t_{i_n})} = \\ &= \frac{n}{3}(t_{i_1} + t_{i_2} + t_{i_3}) + \left(\frac{n}{3} - 1\right)(t_{i_4} + t_{i_5} + t_{i_6}) + \left(\frac{n}{3} - 2\right)(t_{i_7} + t_{i_8} + t_{i_9}) + \dots + 1 \cdot (t_{i_{n-2}} + t_{i_{n-1}} + t_{i_n}) = \\ &= \sum_{j=1}^n \left(\frac{n}{3} - \left\lfloor \frac{j-1}{3} \right\rfloor\right) t_{i_j} = \sum_{j=1}^n \left\lceil \frac{n+1-j}{3} \right\rceil \cdot t_{i_j} \end{aligned}$$

Lemat 3.14. Niech $A_1 \geq A_2 \geq \dots \geq A_n$. Dla dowolnej permutacji $\Pi = (i_1, \dots, i_n)$ zbioru $[n]$ niech $f(\Pi) = \sum_{j=1}^n A_j \cdot t_{i_j}$. Wtedy f osiąga minimum na każdej permutacji ustawiającej czasy niemalejąco, tzn. takiej, że $t_{i_1} \leq t_{i_2} \leq \dots \leq t_{i_n}$.

Wniosek 3.15. Nasze algorytmy szeregowania zadań są poprawne.

Matroidy

Definicja 3.16. Matroid to para $M = (S, \mathcal{A})$ taka, że:

1. S – niepusty zbiór skończony,
2. $\mathcal{A} \subseteq 2^S$ i $\mathcal{A} \neq \emptyset$ – czyli $\mathcal{A}$ to niepusta rodzina podzbiorów S ,
3. $\forall A, B \subseteq S \quad A \in \mathcal{A} \wedge B \subseteq A \implies B \in \mathcal{A}$,
4. własność wymiany:

$$\forall A, B \subseteq S \quad A, B \in \mathcal{A} \wedge |B| > |A| \implies \text{istnieje } x \in B \setminus A \text{ taki, że } A \cup \{x\} \in \mathcal{A}.$$

$\mathcal{A}$ – rodzina zbiorów niezależnych matroidu M , elementy $\mathcal{A}$ – zbiory niezależne matroidu M .

Przykład 3.17.

1. A – macierz, S – zbiór wierszy A , $\mathcal{A} = \{R \subseteq S : R \text{ tworzy zbiór liniowo niezależny}\}$.
Wtedy $(S, \mathcal{A})$ to matroid.
2. $G = (V, E)$ – graf, $\mathcal{A} = \{F \subseteq E : G[F] \text{ jest acykliczny}\}$.
Wtedy $(E, \mathcal{A})$ to matroid zwany matroidem grafowym.

Z definicji $\emptyset \in \mathcal{A}$.

Lemat 3.18. Niech $M = (S, \mathcal{A})$. Wszystkie maksymalne zbiory niezależne M mają tę samą liczbę.

Problem 3.19. $M = (S, \mathcal{A})$ – matroid, $w : S \rightarrow \mathbb{R}_+$ – funkcja wagi.

Dla $A \subseteq S$ niech $w(A) = \sum_{x \in A} w(x)$ – waga A .

Znaleźć zbiór niezależny o maksymalnej wadze w M .

$S = \{x_1, x_2, \dots, x_n\}$.

Algorytm 12: Algorytm Greedy

```

function GREEDY( $M, w$ )
1:   $n \leftarrow |S|$ 
2:   $A \leftarrow \emptyset$   $\triangleright A$  – rozwiązanie
3:  posortuj  $S$  w porządku nierosnącym względem wagi, tak żeby  $w(x_1) \geq w(x_2) \geq \dots \geq w(x_n)$ 
4:  for  $i \leftarrow 1$  to  $n$  do
5:    if  $A \cup \{x_i\} \in \mathcal{A}$  then
6:       $A \leftarrow A \cup \{x_i\}$ 
7:  return  $A$ 

```

Twierdzenie 3.20 (Rado, Edmonds).

Algorytm Greedy znajduje zbiór niezależny o maksymalnej wadze. Co więcej, zachłanny wybór daje optimum dla każdej funkcji wagi wtedy i tylko wtedy, gdy $(S, \mathcal{A})$ jest matroidem.

Złożoność czasowa algorytmu Greedy $n = |S|$

- linia 1: $\mathcal{O}(1)$
- linia 2: $\mathcal{O}(1)$
- linia 3: $\mathcal{O}(n \log n)$
- liczba iteracji pętli z linii 4 wynosi n
- linia 5: $f(n)$ – czas sprawdzenia warunku z linii 5
- linia 6: $\mathcal{O}(1)$
- linia 7: $\mathcal{O}(1)$

Pętla wykonuje n iteracji, a każda kosztuje $f(n) + \mathcal{O}(1)$, co daje koszt pętli $\mathcal{O}(n \cdot f(n))$. Łącznie z sortowaniem otrzymujemy

$$\mathcal{O}(n \log n + n \cdot f(n)),$$

gdzie $f(n)$ to czas sprawdzenia niezależności $A \cup \{x_i\} \in \mathcal{A}$ (linia 5). Złożoność zależy więc od reprezentacji matroidu: dla matroidów, w których test niezależności jest wielomianowy, cały algorytm działa w czasie wielomianowym.

Problem szeregowania zadań

- $S = \{z_1, z_2, \dots, z_n\}$ – zbiór zadań o jednostkowym czasie wykonania
- $d_1, d_2, \dots, d_n$ – deadliney, $d_i \in \{1, \dots, n\}$ dla $i \in [n]$
- $w_1, w_2, \dots, w_n$ – kary, $w_i \geq 0$ dla $i \in [n]$

Płacimy karę w_i za zadanie z_i , jeżeli nie jest skończone w momencie d_i .

Problem 3.21 (szeregowania zadań z karami).

Znaleźć kolejność wykonania zadań minimalizującą sumę kar.

Założmy, że mamy pewien harmonogram. Zadanie z_i nazywamy terminowym w tym harmonogramie, jeśli jest zakończone do chwili d_i , w przeciwnym razie nazywamy je spóźnionym.

Nie płacimy kar za zadania terminowe, płacimy kary za zadania spóźnione.

Lemat 3.22. *Każdy harmonogram można przetransformować na harmonogram z taką samą sumą kar, w którym wszystkie zadania terminowe poprzedzają wszystkie zadania spóźnione oraz zadania terminowe są posortowane w kolejności niemalejącej względem deadlineów.*

Przypomnijmy, że $S = \{z_1, \dots, z_n\}$ to zbiór wszystkich n zadań, czyli $n = |S|$.

Zbiór $A \subseteq S$ nazywamy wolnym od kar, jeśli istnieje harmonogram zbioru A , w którym wszystkie zadania są terminowe.

Dla $t \in \{0, 1, \dots, n\}$ niech

$$N_t(A) = |\{z_i \in A : d_i \leq t\}|$$

– liczba zadań w A o deadline $\leq t$.

Lemat 3.23. Niech $A \subseteq S$. Następujące warunki są równoważne:

1. A jest wolny od kar.
2. $N_t(A) \leq t$ dla każdego $t \in \{1, 2, \dots, n\}$.
3. Jeśli zadania z A są posortowane w kolejności niemalejącej ze względu na deadline, to żadne zadanie nie jest spóźnione.

Twierdzenie 3.24. Niech S – zbiór zadań o jednostkowym czasie wykonania, a $\mathcal{A}$ – rodzina zbiorów wolnych od kar. Wtedy $(S, \mathcal{A})$ jest matroidem.

Dowód. Sprawdzamy własności z definicji matroidu (Definicja 3.16):

(wł. 1) Oczywiście. ✓

(wł. 2) Dla każdego $i \in [n]$ zachodzi $\{z_i\} \in \mathcal{A}$, więc $\mathcal{A} \neq \emptyset$. ✓

(wł. 3) Niech $A \in \mathcal{A}$ (czyli A jest wolny od kar) i $B \subseteq A$. Z definicji istnieje harmonogram zbioru A , w którym wszystkie zadania są terminowe. Usuwając z tego harmonogramu zadania należące do $A \setminus B$, otrzymujemy harmonogram zbioru B , w którym wszystkie zadania nadal są terminowe (usunięcie zadań nie opóźnia pozostałych). Zatem B jest wolny od kar, czyli $B \in \mathcal{A}$.

(wł. 4) Niech $A, B \in \mathcal{A}$, $|B| > |A|$.

Niech $k \in \{0, 1, \dots, n\}$ będzie największą liczbą t taką, że $N_t(B) \leq N_t(A)$.

k jest dobrze zdefiniowane, bo $N_0(B) = 0 = N_0(A)$.

Mamy $N_n(B) = |B| > |A| = N_n(A)$, więc $k < n$.

Dla $j \in \{k+1, k+2, \dots, n\}$ mamy $N_j(B) > N_j(A)$, więc $N_j(B) \geq N_j(A) + 1$.

$$\left. \begin{array}{l} N_{k+1}(B) > N_{k+1}(A) \\ N_k(B) \leq N_k(A) \end{array} \right\} \implies \text{istnieje zadanie } z \in B \setminus A \text{ o deadline } = k+1$$

Niech $A' = A \cup \{z\}$. Pokażemy, że A' jest wolny od kar, używając lematu 3.23.

- Dla $t \in \{1, 2, \dots, k\}$: $N_t(A') = N_t(A) \leq t$, bo A jest wolny od kar.
- Dla $t \in \{k+1, k+2, \dots, n\}$: $N_t(A') = N_t(A) + 1 \leq N_t(B) \leq t$, bo B jest wolny od kar.

Zatem A' jest wolny od kar.

Pokazaliśmy, że $(S, \mathcal{A})$ jest matroidem. □

Możemy do rozwiązania problemu szeregowania zadań użyć algorytmu Greedy (Algorytm 12) dla matroidu $(S, \mathcal{A})$ maksymalizującego $\sum_{z_i \in A} w_i$ (czyli minimalizującego sumę kar zadań spóźnionych).

Z twierdzenia 3.20 (Rado, Edmonds) algorytm Greedy użyty do tego problemu zwraca rozwiązanie optymalne. Warunek z linii 5 algorytmu Greedy (tzn. czy $A \cup \{x_i\} \in \mathcal{A}$) można sprawdzić w czasie $\mathcal{O}(n)$ – lemat 3.23, warunek 3.

Złożoność czasowa algorytmu wynosi:

$$\mathcal{O}(n \log n + n \cdot n) = \mathcal{O}(n^2)$$

Przykład 3.25 (Szeregowanie zadań z deadlineami).

Rozważmy zadania o następujących deadlineach i karach:

i	1	2	3	4	5	6	7	8
d_i	3	2	2	3	1	4	6	5
w_i	80	70	60	50	40	30	20	10

Zadania są już posortowane w porządku nierosnącym względem wagi ($w_1 \geq w_2 \geq \dots \geq w_8$). Kolejne iteracje algorytmu Greedy (poniżej A wypisane w kolejności niemalejącej po deadlineach; po prawej profil $N(A) = (N_1, \dots, N_8)$, który musi spełniać $N_t \leq t$, czyli nie przekraczać $(1, 2, \dots, 8)$):

- | | |
|---|-----------------------------------|
| 1. $A = (z_1)$ | $N(A) = (0, 0, 1, 1, 1, 1, 1, 1)$ |
| 2. $A = (z_2, z_1)$ | $N(A) = (0, 1, 2, 2, 2, 2, 2, 2)$ |
| 3. $A = (z_2, z_3, z_1)$ | $N(A) = (0, 2, 3, 3, 3, 3, 3, 3)$ |
| 4. $A = (z_2, z_3, z_1)$ (z_4 nie dodajemy, bo $N_3(A \cup \{z_4\}) = 4 > 3$) | |
| 5. $A = (z_2, z_3, z_1)$ (z_5 nie dodajemy, bo $N_2(A \cup \{z_5\}) = 3 > 2$) | |
| 6. $A = (z_2, z_3, z_1, z_6)$ | $N(A) = (0, 2, 3, 4, 4, 4, 4, 4)$ |
| 7. $A = (z_2, z_3, z_1, z_6, z_7)$ | $N(A) = (0, 2, 3, 4, 4, 5, 5, 5)$ |
| 8. $A = (z_2, z_3, z_1, z_6, z_8, z_7)$ | $N(A) = (0, 2, 3, 4, 5, 6, 6, 6)$ |

Harmonogram (zadania terminowe posortowane po deadlineach, potem zadania spóźnione):

$(z_2, z_3, z_1, z_6, z_8, z_7, z_4, z_5)$

Suma kar = $w_4 + w_5 = 50 + 40 = 90$

Algorytmy aproksymacyjne

Algorytmy aproksymacyjne stosuje się do problemów, dla których jest mała szansa na znalezienie algorytmu dokładnego (np. wielomianowego). Zamiast szukać rozwiązania optymalnego, szukamy rozwiązania „prawie optymalnego”.

Niech Π – problem optymalizacyjny znalezienia maksimum lub minimum pewnej funkcji celu f .

- D_Π – zbiór instancji problemu Π .
- $I \in D_\Pi$ – instancja Π .
- $S(I)$ – zbiór rozwiązań dopuszczalnych dla instancji I .
- $f : \bigcup_{I \in D_\Pi} S(I) \rightarrow \mathbb{R}$ – funkcja celu. Zakładamy $f(s) > 0$ dla każdego rozwiązania dopuszczalnego (potrzebne, by iloraz jakości miał sens; dla typowych problemów koszt jest z natury dodatni).

Niech $c^*(I)$ oznacza wartość rozwiązania optymalnego instancji I , $c^*(I) > 0$.

A – algorytm aproksymacyjny rozwiązujący Π .

$c(I)$ – wartość rozwiązania znalezionej przez algorytm A dla instancji I , $c(I) > 0$.

Definicja 4.1. Mówimy, że algorytm A ma współczynnik aproksymacji $\rho(n)$, jeśli dla każdej instancji $I \in D_\Pi$ rozmiaru n zachodzi

$$\max \left\{ \frac{c(I)}{c^*(I)}, \frac{c^*(I)}{c(I)} \right\} \leq \rho(n)$$

Wtedy A nazywamy algorytmem $\rho(n)$ -aproksymacyjnym.

Zauważmy, że:

$$\left. \begin{array}{l} \text{dla problemu maksymalizacji: } 0 < c(I) \leq c^*(I) \\ \text{dla problemu minimalizacji: } 0 < c^*(I) \leq c(I) \end{array} \right\} \Rightarrow \max \left\{ \frac{c(I)}{c^*(I)}, \frac{c^*(I)}{c(I)} \right\} \geq 1$$

Niech $\rho = \sup_{n \in \mathbb{N}} \{\rho(n)\}$. Wtedy mówimy, że współczynnik aproksymacji algorytmu A wynosi ρ i A nazywamy algorytmem ρ -aproksymacyjnym.

Problem pokrycia wierzchołkowego

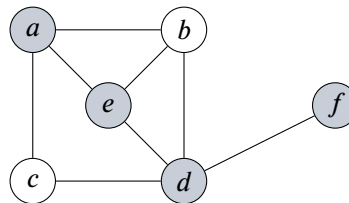
Definicja 4.2. Niech $G = (V, E)$ – graf. Pokryciem wierzchołkowym (Vertex Cover, VC) grafu G nazywamy zbiór $W \subseteq V$ taki, że dla każdej krawędzi $e \in E$ zachodzi $W \cap e \neq \emptyset$ – czyli każda krawędź G ma co najmniej jeden koniec w W .

Problem 4.3 (pokrycia wierzchołkowego).

- Instancja: graf G .
- Problem: znaleźć pokrycie wierzchołkowe grafu G o minimalnej liczbie wierzchołków.

Przykład 4.4 (Pokrycie wierzchołkowe).

Rozważmy graf G przedstawiony poniżej:



$W = \{a, e, d, f\}$ jest pokryciem wierzchołkowym powyższego grafu G (zaznaczone wyróżnione wierzchołki).

Algorytm Approx Vertex Cover

Algorytm 13: Approx Vertex Cover

```

function AVC( $G$ )
1:   $C \leftarrow \emptyset$ 
2:   $F \leftarrow E(G)$ 
3:  while  $F \neq \emptyset$  do
4:    wybierz dowolną krawędź  $uv \in F$ 
5:     $C \leftarrow C \cup \{u, v\}$ 
6:    usuń z  $F$  wszystkie krawędzie incydentne z  $u$  lub z  $v$ 
7:  return  $C$ 
    
```

▷ C – rozwiązanie
▷ F – zbiór dotychczas niepokrytych krawędzi

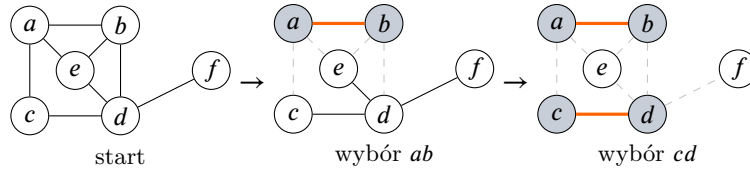
Przykład 4.5 (Algorytm AVC – przykład).

Rozważmy graf z poprzedniego przykładu.

Pierwszy przebieg algorytmu może wybrać kolejno krawędzie ab i cd :

- Po wybraniu ab usuwamy z F krawędzie ab, ac, ae, bd, be , zostają cd, de, df .
- Po wybraniu cd usuwamy z F krawędzie cd, de, df – $F = \emptyset$.

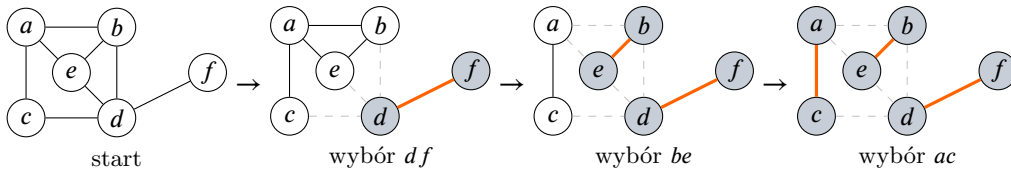
Algorytm zwraca $C = \{a, b, c, d\}$.



Inny przebieg algorytmu może wybrać kolejno krawędzie df , be i ac :

- Po wybraniu df usuwamy z F krawędzie df, de, bd, cd .
- Po wybraniu be usuwamy z F krawędzie be, ae, ab .
- Po wybraniu ac usuwamy ostatnią krawędź $ac - F = \emptyset$.

Algorytm zwraca $C = \{d, f, e, b, a, c\}$.



Złożoność czasowa algorytmu AVC $n = |V(G)|$, $m = |E(G)|$.

- linia 1: $\mathcal{O}(1)$
- linia 2: $\mathcal{O}(m)$
- pętlę z linii 3–6 można zaimplementować, żeby działała w czasie $\mathcal{O}(m)$
- linia 7: $\mathcal{O}(1)$

Złożoność czasowa AVC wynosi $\mathcal{O}(m)$.

Twierdzenie 4.6. *AVC jest algorytmem 2-aproksymacyjnym.*

Dowód. Niech G – graf wejściowy.

$c^*(G)$ – najmniejsza liczba wierzchołków w pokryciu wierzchołkowym G .

$c(G)$ – liczba wierzchołków w pokryciu wierzchołkowym znalezionym przez AVC.

Niech B będzie zbiorem krawędzi wybranych przez AVC w linii 4.

B jest skojarzeniem (czyli zbiorem parami rozłącznych krawędzi), bo po wybraniu krawędzi uv algorytm usuwa z F wszystkie krawędzie incydentne z u lub v .

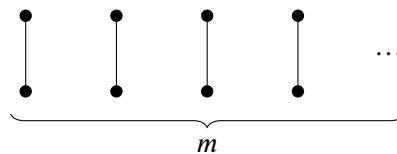
Żeby pokryć krawędzie z B potrzeba co najmniej $|B|$ wierzchołków. Stąd $c^*(G) \geq |B|$.

$c(G) = 2|B|$, bo algorytm dodaje do C oba końce każdej krawędzi z B .

$$c(G) = 2|B| \leq 2 \cdot c^*(G) \implies \frac{c(G)}{c^*(G)} \leq 2$$

□

Pokażemy, że ograniczenie 2 jest osiągalne. Rozważmy graf złożony z m rozłącznych krawędzi:



AVC zwraca pokrycie złożone z $2m$ wierzchołków, natomiast optymalne pokrycie ma rozmiar m (po jednym końcu z każdej krawędzi). Stąd $\frac{c(G)}{c^*(G)} = 2$.

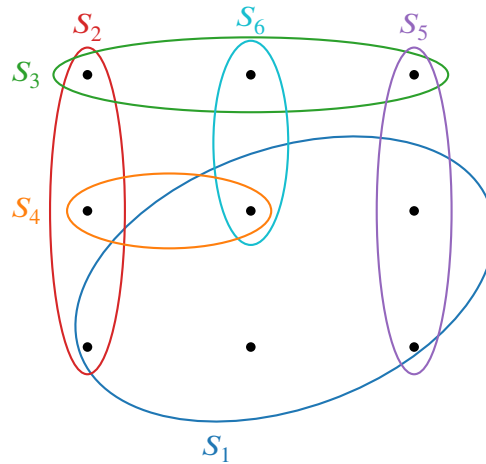
Problem pokrycia zbioru

Definicja 4.7. Niech X – zbiór skończony, $|X| = n$, $\mathcal{F} \subseteq 2^X$ – rodzina podzbiorów zbioru X taka, że $\bigcup_{S \in \mathcal{F}} S = X$ (każdy element X należy do co najmniej jednego zbioru z $\mathcal{F}$).

Rodzina $\mathcal{A} \subseteq \mathcal{F}$ jest pokryciem zbioru X , jeśli $\bigcup_{S \in \mathcal{A}} S = X$.

Przykład 4.8 (Pokrycie zbioru).

Rozważmy zbiór X i rodzinę podzbiorów przedstawione poniżej:



$\mathcal{F} = \{S_1, S_2, S_3, S_4, S_5, S_6\}$. $\{S_1, S_2, S_3\}$ – pokrycie.

Problem 4.9 (pokrycia zbioru).

- Instancja: $(X, \mathcal{F})$, gdzie X – zbiór skończony, $|X| = n$, $\mathcal{F} \subseteq 2^X$ taki, że $\bigcup_{S \in \mathcal{F}} S = X$.
- Problem: znaleźć pokrycie $\mathcal{A} \subseteq \mathcal{F}$ minimalizujące $|\mathcal{A}|$.

Uwaga. VC jest szczególnym przypadkiem problemu pokrycia zbiorów. Dla $G = (V, E)$ kładziemy $X = E$, $\mathcal{F} = \{E(v) : v \in V\}$, gdzie $E(v) = \{e \in E : v \in e\}$.

Algorytm Greedy Set Cover

Algorytm 14: Greedy Set Cover

```

function GSC( $X, \mathcal{F}$ )
1:    $U \leftarrow X$ 
2:    $\mathcal{A} \leftarrow \emptyset$ 
3:   while  $U \neq \emptyset$  do
4:     wybierz  $S \in \mathcal{F}$  maksymalizujący  $|S \cap U|$ 
5:      $\mathcal{A} \leftarrow \mathcal{A} \cup \{S\}$ 
6:      $U \leftarrow U \setminus S$ 
7:   return  $\mathcal{A}$ 

```

$\triangleright U$ – zbiór jeszcze niepokrytych elementów
 $\triangleright \mathcal{A}$ – rozwiązanie

Poprawność Pętla wykonuje się, dopóki istnieją niepokryte elementy, a w każdej iteracji co najmniej jeden z nich zostaje pokryty, więc po skończonej liczbie kroków U pustoszeje.

Złożoność czasowa

- linia 1: $\mathcal{O}(n)$
- linia 2: $\mathcal{O}(1)$
- liczba iteracji pętli 3–6 jest $\leq |X|$ i $\leq |\mathcal{F}|$, więc jest $\leq \min\{|X|, |\mathcal{F}|\}$. Wnętrze pętli:
 - linia 4: $\mathcal{O}(|X| \cdot |\mathcal{F}|)$
 - linia 5: $\mathcal{O}(1)$
 - linia 6: $\mathcal{O}(|X|)$
 łącznie $\mathcal{O}(|X| \cdot |\mathcal{F}|)$.
- linia 7: $\mathcal{O}(1)$

Ostatecznie złożoność GSC wynosi $\mathcal{O}(|X| \cdot |\mathcal{F}| \cdot \min\{|X|, |\mathcal{F}|\})$.

Twierdzenie 4.10. *GSC jest algorytmem $H(\max\{|S| : S \in \mathcal{F}\})$ -aproxymacyjnym, gdzie*

$$H(0) = 0, \quad H(m) = 1 + \frac{1}{2} + \frac{1}{3} + \dots + \frac{1}{m} \text{ dla } m \geq 1$$

Dowód. Niech $(X, \mathcal{F})$ będzie instancją wejściową.

$\mathcal{A}^*$ – optymalne pokrycie dla $(X, \mathcal{F})$.

$\mathcal{A}$ – pokrycie dla $(X, \mathcal{F})$ znalezione przez GSC.

Niech S_i będzie zbiorem wybranym przez algorytm w linii 4 w i -tej iteracji pętli. Przyznajemy koszt

$$c_x = \frac{1}{|S_i \setminus (S_1 \cup S_2 \cup \dots \cup S_{i-1})|}$$

każdemu elementowi $x \in S_i \setminus (S_1 \cup \dots \cup S_{i-1})$ ($S_i \setminus (S_1 \cup \dots \cup S_{i-1})$ to zbiór elementów pokrytych po raz pierwszy w i -tej iteracji pętli). Całkowity koszt przyznany podczas jednej iteracji wynosi 1, a ponieważ $|\mathcal{A}|$ jest liczbą iteracji pętli, to

$$|\mathcal{A}| = \sum_{x \in X} c_x$$

Rozważmy sumę $\sum_{S \in \mathcal{A}^*} \sum_{x \in S} c_x$. Skoro $\mathcal{A}^*$ jest pokryciem, każdy $x \in X$ występuje w tej sumie co najmniej raz, więc

$$\sum_{S \in \mathcal{A}^*} \sum_{x \in S} c_x \geq \sum_{x \in X} c_x = |\mathcal{A}|$$

Pokażemy, że $\sum_{x \in S} c_x \leq H(|S|)$ dla każdego $S \in \mathcal{F}$.

Niech $S \in \mathcal{F}$. Dla $i \in \{1, 2, \dots, |\mathcal{A}|\}$ niech $n_i = |S \setminus (S_1 \cup \dots \cup S_i)|$ (liczba elementów w S , które nie są pokryte po i -tej iteracji pętli) i niech $n_0 = |S|$.

Niech k będzie najmniejszą liczbą taką, że $n_k = 0$. Wtedy S jest pokryty przez $S_1 \cup S_2 \cup \dots \cup S_k$. Oczywiście $n_{i-1} \geq n_i$ dla $i \in [k]$, a $n_{i-1} - n_i$ to liczba elementów z S pokrytych po raz pierwszy przez S_i .

Zatem

$$\sum_{x \in S} c_x = \sum_{i=1}^k (n_{i-1} - n_i) \cdot \frac{1}{|S_i \setminus (S_1 \cup \dots \cup S_{i-1})|}$$

Na podstawie wyboru zachłannego w linii 4 mamy $|S_i \setminus (S_1 \cup \dots \cup S_{i-1})| \geq |S \setminus (S_1 \cup \dots \cup S_{i-1})| = n_{i-1}$ dla każdego $S \in \mathcal{F} \setminus \{S_1, \dots, S_{i-1}\}$.

Mamy:

$$\begin{aligned}
\sum_{x \in S} c_x &= \sum_{i=1}^k (n_{i-1} - n_i) \cdot \frac{1}{|S_i \setminus (S_1 \cup \dots \cup S_{i-1})|} \leq \sum_{i=1}^k (n_{i-1} - n_i) \cdot \frac{1}{n_{i-1}} \\
&= \sum_{i=1}^k \sum_{j=n_i+1}^{n_{i-1}} \frac{1}{n_{i-1}} \leq \sum_{i=1}^k \sum_{j=n_i+1}^{n_{i-1}} \frac{1}{j} = \sum_{i=1}^k \left(\sum_{j=1}^{n_{i-1}} \frac{1}{j} - \sum_{j=1}^{n_i} \frac{1}{j} \right) \\
&= \sum_{i=1}^k (H(n_{i-1}) - H(n_i)) \\
&= (H(n_0) - H(n_1)) + (H(n_1) - H(n_2)) + \dots + (H(n_{k-1}) - H(n_k)) \\
&= H(n_0) - H(n_k) = H(|S|) - H(0) = H(|S|).
\end{aligned}$$

Ostatecznie mamy:

$$\begin{aligned}
|\mathcal{A}| &= \sum_{x \in X} c_x \leq \sum_{S \in \mathcal{A}^*} \sum_{x \in S} c_x \leq \sum_{S \in \mathcal{A}^*} H(|S|) \leq \sum_{S \in \mathcal{A}^*} \max\{H(|S|) : S \in \mathcal{A}^*\} \\
&= |\mathcal{A}^*| \cdot \max\{H(|S|) : S \in \mathcal{A}^*\} \\
&\leq |\mathcal{A}^*| \cdot \max\{H(|S|) : S \in \mathcal{F}\} = |\mathcal{A}^*| \cdot H(\max\{|S| : S \in \mathcal{F}\}),
\end{aligned}$$

więc $\frac{|\mathcal{A}|}{|\mathcal{A}^*|} \leq H(\max\{|S| : S \in \mathcal{F}\})$. □

Wniosek 4.11. *GSC jest algorytmem $(\ln |X| + 1)$ -aproksymacyjnym.*

Dowód. $H(n) \leq \ln n + 1$, więc

$$H(\max\{|S| : S \in \mathcal{F}\}) \leq H(|X|) \leq \ln |X| + 1$$

□

VC jest szczególnym przypadkiem problemu pokrycia zbiorów. Dla $G = (V, E)$ kładziemy $X = E$, $\mathcal{F} = \{E(v) : v \in V\}$, gdzie $E(v) = \{e \in E : v \in e\}$, $|E(v)| = d(v)$. Wtedy $\max\{|E(v)| : E(v) \in \mathcal{F}\} = \Delta(G)$, więc GSC zastosowany do VC ma współczynnik aproksymacji $H(\Delta(G))$.

Dla grafów o maksymalnym stopniu ≤ 3 ($H(3) < 2 < H(4)$) oszacowanie współczynnika aproksymacyjnego GSC jest lepsze niż oszacowanie dla AVC.

Schematy aproksymacyjne

Niech Π – problem (NP-trudny).

Definicja 4.12. Algorytm A jest schematem aproksymacyjnym dla Π , jeśli dla dowolnego wejścia (I, ϵ) , gdzie I jest instancją Π i $\epsilon > 0$, A jest algorytmem $(1 + \epsilon)$ -aproksymacyjnym.

Definicja 4.13. Schemat aproksymacyjny A nazywamy wielomianowym schematem aproksymacyjnym, jeśli jego złożoność czasowa może być ograniczona wielomianową funkcją względem $|I|$ dla każdego ustalonego $\varepsilon > 0$.

Złożoność czasowa wielomianowego schematu aproksymacyjnego może wynosić np. $\Theta(2^{1/\varepsilon} \cdot n^2)$, gdzie $n = |I|$. Wtedy zmniejszanie precyzji (czyli zmniejszanie ε) jest kosztowne czasowo.

Definicja 4.14. Schemat aproksymacyjny A nazywamy w pełni wielomianowym schematem aproksymacyjnym, jeśli jego złożoność czasowa może być ograniczona funkcją wielomianową względem $|I|$ i $\frac{1}{\varepsilon}$.

Problem sumy podzbiorów

Problem 4.15 (sumy podzbiorów (Subset-Sum)).

Instancja: $S = \{x_1, x_2, \dots, x_n\} \subseteq \mathbb{N}$, $t \in \mathbb{N}$.

- *Wersja decyzyjna*: czy istnieje $S' \subseteq S$ takie, że $\sum_{x_i \in S'} x_i = t$?
- *Wersja optymalizacyjna*: znaleźć $S' \subseteq S$ takie, że $\sum_{x_i \in S'} x_i$ jest największą możliwą liczbą $\leq t$.

Dla zbioru $P \subseteq \mathbb{N}$ i $x \in \mathbb{N}$ oznaczamy $P + x = \{s + x : s \in P\}$.

np. dla $P = \{1, 2, 3, 6\}$ i $x = 2$: $P + x = \{3, 4, 5, 8\}$.

Dla listy L liczb naturalnych i $x \in \mathbb{N}$ przez $L + x$ oznaczamy listę powstałą z L przez dodanie x do każdego elementu.

np. $L = (3, 5, 8)$ i $x = 3$: $L + x = (6, 8, 11)$.

Lemat 4.16. Niech $S = \{x_1, \dots, x_n\} \subseteq \mathbb{N}$. Niech $P_0 = \{0\}$ i $P_i = P_{i-1} \cup (P_{i-1} + x_i)$ dla $i \in [n]$. Zbiór P_n składa się ze wszystkich możliwych sum podzbiorów zbioru S .

Lemat 4.16 podsuwa algorytm: wystarczy liczyć kolejne zbiory P_i . Reprezentujemy je posortowanymi listami L_i bez duplikatów, a rekurencję $P_i = P_{i-1} \cup (P_{i-1} + x_i)$ realizujemy scalaniem list. Dodatkowo z L_i usuwamy elementy $> t$ – żadna suma większa od t nie może być rozwiązaniem, więc nie wpływa to na wynik.

Założmy, że mamy procedurę MERGELists(L, L'), która dla posortowanych list liczb naturalnych L, L' zwraca posortowaną listę elementów z L i L' i usuwa duplikaty. Złożoność czasowa MERGELists(L, L') wynosi $\Theta(|L| + |L'|)$.

Algorytm 15: Exact Subset Sum

```

function EXACTSUBSETSUM( $S, t$ )
1:   $n \leftarrow |S|$ 
2:   $L_0 \leftarrow (0)$ 
3:  for  $i \leftarrow 1$  to  $n$  do
4:     $L_i \leftarrow \text{MERGELists}(L_{i-1}, L_{i-1} + x_i)$ 
5:    usuń z  $L_i$  wszystkie liczby  $> t$ 
6:  return największy element  $L_n$ 

```

Poprawność: indukcyjnie L_i to posortowana lista tych elementów P_i , które są $\leq t$. Na mocy lematu 4.16 P_n zawiera wszystkie sumy podzbiorów S , więc największy element L_n jest największą taką sumą $\leq t$ – czyli rozwiązaniem wersji optymalizacyjnej.

Złożoność czasowa Długość listy L_i może być wykładnicza względem i , nawet 2^i . Dla $I = (S, t)$:

$$|I| = \Theta\left(\log t + \sum_{i=1}^n \log x_i\right)$$

Jeśli liczby t i x_i są n -cyfrowe, to $\log t, \log x_i = \Theta(n)$ i wtedy $|I| = \Theta(n + n^2) = \Theta(n^2)$. Skoro L_n może mieć długość 2^n , to EXACTSUBSETSUM (Algorytm 15) jest algorytmem wykładniczym.

Jeśli liczba t jest ograniczona przez funkcję wielomianową od $n = |S|$, czyli $t = \mathcal{O}(n^p)$ dla jakiegoś $p \in \mathbb{N}$, to długość L_i jest ograniczona przez $\mathcal{O}(t) = \mathcal{O}(n^p)$ i w tym przypadku algorytm działa w czasie wielomianowym.

Schemat aproksymacyjny Pokażemy w pełni wielomianowy schemat aproksymacyjny dla problemu sumy podzbiorów (Problem 4.15).

Pomysł: będziemy skracać listy L_i . Dla $0 < \delta < 1$ z listy L robimy krótszą listę L' taką, że dla każdego elementu y usuniętego z L istnieje element z w L' , który jest blisko niego, dokładnie $\frac{y}{1+\delta} \leq z \leq y$.

Przykład 4.17 (Procedura Trim).

$L = (15, 16, 17, 25, 27, 28, 35, 36, 37)$, $\delta = 0,1$ (czyli próg $1 + \delta = 1,1$).

Idziemy po liście od lewej; ostatni zachowany element nazwijmy z . Kolejny y zachowujemy tylko wtedy, gdy $y > z \cdot 1,1$ – w przeciwnym razie y jest na tyle blisko z , że z go reprezentuje ($\frac{y}{1,1} \leq z \leq y$).

- 15 – pierwszy element, zachowujemy; $z = 15$.
- $16 \leq 15 \cdot 1,1 = 16,5$ – usuwamy, reprezentuje je 15.
- $17 > 16,5$ – zachowujemy; $z = 17$.
- $25 > 17 \cdot 1,1 = 18,7$ – zachowujemy; $z = 25$.
- $27 \leq 25 \cdot 1,1 = 27,5$ – usuwamy, reprezentuje je 25.
- $28 > 27,5$ – zachowujemy; $z = 28$.
- $35 > 28 \cdot 1,1 = 30,8$ – zachowujemy; $z = 35$.
- $36 \leq 35 \cdot 1,1 = 38,5$ – usuwamy, reprezentuje je 35.
- $37 \leq 38,5$ – usuwamy, reprezentuje je 35.

Stąd $L' = (15, 17, 25, 28, 35)$.

Następująca procedura $\text{TRIM}(L, \delta)$ realizuje ten pomysł.

$L = (y_1, \dots, y_m)$ – posortowana rosnąco lista, $0 < \delta < 1$.

Algorytm 16: Trim

```

function TRIM( $L, \delta$ )
1:   $m \leftarrow |L|$ 
2:   $L' \leftarrow (y_1)$ 
3:   $last \leftarrow y_1$ 
4:  for  $i \leftarrow 2$  to  $m$  do
5:    if  $y_i > last \cdot (1 + \delta)$  then
6:      dołącz  $y_i$  na koniec  $L'$ 
7:       $last \leftarrow y_i$ 
8:  return  $L'$ 

```

Czas działania $\text{TRIM}(L, \delta)$ wynosi $\Theta(|L|)$.

Oto schemat aproksymacyjny:

Algorytm 17: Approx Subset Sum

```

function APPROXSUBSETSUM( $S, t, \epsilon$ )
1:   $n \leftarrow |S|$ 
2:   $L_0 \leftarrow (0)$ 
3:  for  $i \leftarrow 1$  to  $n$  do
4:     $L_i \leftarrow \text{MERGE}(\text{LISTS}(L_{i-1}, L_{i-1} + x_i))$ 
5:    usuń z  $L_i$  liczby  $> t$ 
6:     $L_i \leftarrow \text{TRIM}(L_i, \frac{\epsilon}{2n})$ 
7:  return największy element  $z \in L_n$ 

```

$\triangleright \delta = \frac{\epsilon}{2n}$
 $\triangleright z^*$

Problem sumy podzbioru

Analizujemy schemat APPROXSUBSETSUM (Algorytm 17) z procedurą TRIM (Algorytm 16); ustalamy $\delta = \frac{\epsilon}{2n}$.

Przykład 4.18 (Aproksymacja sumy podzbioru).

$S = \{205, 203, 400, 200\}$, $t = 650$, $\epsilon = 0,4$, $n = 4$.

Wtedy $\delta = \frac{\epsilon}{2n} = \frac{0,4}{8} = 0,05$, czyli próg przycinania to $1 + \delta = 1,05$.

Elementy S przetwarzamy w kolejności $x_1 = 205$, $x_2 = 203$, $x_3 = 400$, $x_4 = 200$. W każdym obrocie pętli: scalamy L_{i-1} z $L_{i-1} + x_i$ (linia 4), usuwamy elementy $> t = 650$ (linia 5), a następnie przycinamy listę procedurą TRIM z progiem 1,05 (linia 6). W przycinaniu zachowany element z usuwa kolejne y , gdy $y \leq z \cdot 1,05$.

$$L_0 = (0)$$

$$L_1 = (0, 205)$$

(nic do scalenia, usunięcia ani przycięcia)

$$i = 2, x_2 = 203 :$$

$$L_2 = (0, 203, 205, 408)$$

(linia 4: scalenie z (203, 408))

$$L_2 = (0, 203, 205, 408)$$

(linia 5: brak elementów > 650)

$$L_2 = (0, 203, 408)$$

(linia 6: $205 \leq 203 \cdot 1,05 = 213,15$)

$$i = 3, x_3 = 400 :$$

$$L_3 = (0, 203, 400, 408, 603, 808)$$

(linia 4: scalenie z (400, 603, 808))

$$L_3 = (0, 203, 400, 408, 603)$$

(linia 5: usuwamy 808 > 650)

$$L_3 = (0, 203, 400, 603)$$

(linia 6: $408 \leq 400 \cdot 1,05 = 420$)

$$i = 4, x_4 = 200 :$$

$$L_4 = (0, 200, 203, 400, 403, 600, 603, 803)$$

(linia 4: scalenie z (200, 403, 600, 803))

$$L_4 = (0, 200, 203, 400, 403, 600, 603)$$

(linia 5: usuwamy 803 > 650)

$$L_4 = (0, 200, 400, 600)$$

(linia 6: 203, 403, 603 przycięte)

Algorytm zwraca $z^* = 600$ (bo $600 = 400 + 200$). Optymalna suma $\leq t$ to $y^* = 205 + 203 + 200 = 608$.

Błąd względny mieści się w gwarancji: $\frac{y^*}{z^*} = \frac{608}{600} \approx 1,013 \leq 1,4 = 1 + \epsilon$.

Twierdzenie 4.19. Algorytm APPROXSUBSETSUM jest w pełni wielomianowym schematem aproksymacyjnym dla problemu sumy podzbioru.

Pokażemy, że algorytm działa w czasie wielomianowym względem $|I| = \Theta(\log t + \sum_{i=1}^n \log x_i)$ i $\frac{1}{\epsilon}$.

Na każdej liście L_i po linii 6 elementy $z_0 = 0, z_1, \dots, z_k$ spełniają warunek $z_{j+1} > z_j \left(1 + \frac{\epsilon}{2n}\right)$ dla $j \geq 1$. Ile różnych liczb naturalnych $z_1, \dots, z_k$ spełniających ten warunek może się tu zmieścić w przedziale $[1, t]$?

Przez indukcję po i mamy, że $z_{i+1} > z_1 \left(1 + \frac{\epsilon}{2n}\right)^i$ dla $1 \leq i \leq k-1$. Zatem

$$z_1 \left(1 + \frac{\epsilon}{2n}\right)^{k-1} < z_k \leq t$$

Skoro $z_1 \geq 1$, to (korzystając z $\frac{x}{1+x} \leq \ln(1+x)$ dla $x \geq 0$)

$$\left(1 + \frac{\epsilon}{2n}\right)^{k-1} \leq t \quad \Bigg| \ln$$

$$(k-1) \ln \left(1 + \frac{\epsilon}{2n}\right) \leq \ln t$$

$$k-1 \leq \frac{\ln t}{\ln \left(1 + \frac{\epsilon}{2n}\right)} \quad \Bigg| +2$$

$$k+1 \leq \frac{\ln t}{\ln \left(1 + \frac{\epsilon}{2n}\right)} + 2 \leq \frac{\ln t \cdot \left(1 + \frac{\epsilon}{2n}\right)}{\frac{\epsilon}{2n}} + 2 = \frac{2n + \epsilon}{2n} \cdot \frac{2n}{\epsilon} \ln t + 2 \leq \frac{3n \ln t}{\epsilon} + 2$$

Skoro długość L_i po linii 6 wynosi $k+1 \leq \frac{3n \ln t}{\epsilon} + 2$, a po $(i+1)$ -szej linii 4 może być co najwyżej 2 razy dłuższa i $|I| = \Omega(\ln t + n)$, to złożoność czasowa algorytmu jest wielomianowa względem $|I|$ i $\frac{1}{\epsilon}$.

Skojarzenia w grafach

Najliczniejsze skojarzenie w grafach dwudzielnych $G = (V, E)$ – graf.

G jest dwudzielny jeśli istnieje podział V na dwa niepuste podzbiory V_1, V_2 takie, że dla każdej krawędzi $e = v_1 v_2 \in E$ mamy $v_1 \in V_1$ i $v_2 \in V_2$. Oznaczamy to jako $G = (V_1, V_2, E)$.

Skojarzenie w G to zbiór parami rozłącznych krawędzi.

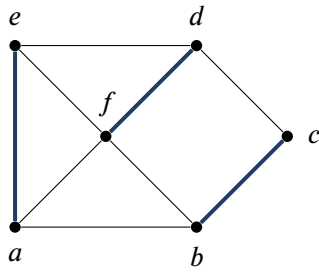
Skojarzenie jest najliczniejsze jeśli ma największą liczbę.

Problem 5.1 (najliczniejszego skojarzenia w grafie dwudzielnym).

Instancja: graf dwudzielny $G = (V_1, V_2; E)$.

Znaleźć: najliczniejsze skojarzenie w G .

Przykład 5.2.



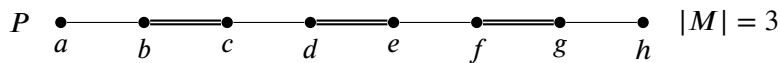
Przykładowe skojarzenie: $\{ae, df, bc\}$

Jeśli M jest skojarzeniem, to każdy wierzchołek należy do co najwyżej jednej krawędzi z M .

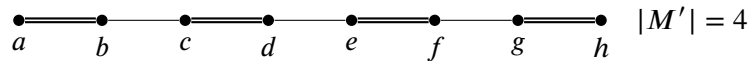
Niech M będzie ustalonym skojarzeniem w G .

- wierzchołek jest skojarzony jeśli należy do jakiejś krawędzi z M (mówimy też, że M pokrywa ten wierzchołek),
- wierzchołek jest wolny jeśli nie jest skojarzony,
- krawędź jest skojarzona jeśli należy do M ,
- krawędź jest wolna jeśli nie należy do M .

Ścieżka powiększająca (względem M) to naprzemienna ścieżka składająca się z wolnych i skojarzonych krawędzi o wolnych końcach.



$$M' = M \div E(P) = (M - E(P)) \cup (E(P) - M)$$



Możemy użyć ścieżki powiększającej i różnicy symetrycznej do zwiększenia liczności skojarzenia.

Twierdzenie 5.3 (Berge'a).

Niech $G = (V, E)$ – graf, M – skojarzenie w G .

M jest najliczniejszym skojarzeniem w $G \iff$ nie istnieje ścieżka powiększająca względem M .

Dowód.

$\implies$ Przypuśćmy, że istnieje ścieżka powiększająca P względem M .

Wtedy $M' = M \div E(P)$ jest skojarzeniem i $|M'| > |M|$, więc M nie jest skojarzeniem najliczniejszym.

$\impliedby$ Przypuśćmy, że M nie jest najliczniejszym skojarzeniem w G .

Niech M' będzie skojarzeniem takim, że $|M'| > |M|$.

Niech H będzie podgrafem G indukowanym przez $M' \div M$, czyli $H = (V(G), M' \div M)$.

M, M' to skojarzenia, więc stopień wierzchołka $d_H(v) \leq 2$ dla każdego $v \in V(G)$.

Każda składowa H jest ścieżką lub cyklem.

M, M' to skojarzenia, więc nie ma cykli nieparzystych w H .

Każda składowa H jest ścieżką lub cyklem parzystym.

Skoro $|M'| > |M|$, to istnieje składowa H , która jest ścieżką zawierającą więcej krawędzi z M' niż z M , więc ta ścieżka zaczyna się i kończy krawędzią z M' . Ta ścieżka jest ścieżką powiększającą względem M .

□

Przedstawimy teraz algorytm znajdujący najliczniejsze skojarzenie w grafie dwudzielnym (Problem 5.1).

Niech M będzie skojarzeniem w G .

Definiujemy graf skierowany $G_M = (V_1 \cup V_2, E_M)$, gdzie

$$E_M = \{(u, v) : uv \in E - M, u \in V_1, v \in V_2\} \cup \{(v, u) : uv \in M, u \in V_1, v \in V_2\}.$$

Innymi słowy, krawędzie wolne kierujemy z V_1 do V_2 , a krawędzie skojarzone z V_2 do V_1 .

Przykład 5.4.



Procedura do znajdowania ścieżki powiększającej — *Augmenting Path Search* (APS).

Algorytm 18: Augmenting Path Search

function APS(G, M)

$\triangleright G = (V_1, V_2; E)$, M – skojarzenie w G

- 1: $V'_1 \leftarrow$ zbiór wierzchołków wolnych z V_1
- 2: $V'_2 \leftarrow$ zbiór wierzchołków wolnych z V_2
- 3: skonstruuj graf skierowany $G_M = (V_1 \cup V_2, E_M)$
- 4: znajdź ścieżkę $\vec{P}$ z V'_1 do V'_2 w G_M
- 5: **if** $\vec{P}$ istnieje **then**
- 6: **return** P
- 7: **else**
- 8: **return** NULL

$\triangleright P$ – nieskierowana $\vec{P}$

Lemat 5.5. *Procedura APS zwraca ścieżkę $P \iff$ istnieje ścieżka powiększająca względem M w G . Ponadto ścieżka P zwrócona przez APS jest ścieżką powiększającą względem M .*

Algorytm znajdowania najliczniejszego skojarzenia — *Maximum Matching* (MM).

Algorytm 19: Maximum Matching

function MM(G)

$\triangleright G = (V_1, V_2; E)$

- 1: $M \leftarrow \emptyset$
- 2: **repeat**
- 3: $P \leftarrow$ APS(G, M)
- 4: **if** $P \neq \text{NULL}$ **then**
- 5: $M \leftarrow M \div E(P)$
- 6: **until** $P = \text{NULL}$
- 7: **return** M

Poprawność Wynika z twierdzenia Berge'a.

Złożoność czasowa Złożoność czasowa APS:

- Linia 1, 2: $\mathcal{O}(|V|)$.
- Linia 3: $\mathcal{O}(|E|)$.
- Linia 4: $\mathcal{O}(|E|)$ (używamy algorytmu BFS).
- Linia 5–8: $\mathcal{O}(|V|)$.

Złożoność APS wynosi zatem $\mathcal{O}(|E|)$.

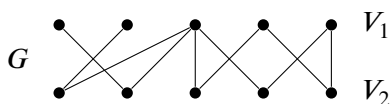
Złożoność czasowa MM:

- Linia 1: $\mathcal{O}(1)$.
- Pętla 2–6 wykona się co najwyżej $\frac{|V|}{2}$ razy, bo w każdej iteracji $|M|$ zwiększa się o 1, a maksymalna liczność skojarzenia jest $\leq \frac{|V|}{2}$.
- Wnętrze pętli:
 - Linia 3: $\mathcal{O}(|E|)$.
 - Linia 5: $\mathcal{O}(|V|)$.
 Razem $\mathcal{O}(|E|)$.
- Linia 7: $\mathcal{O}(|V|)$.

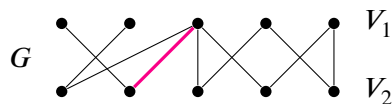
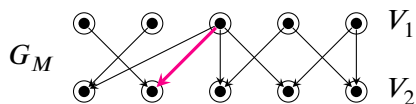
Ostateczna złożoność MM wynosi $\mathcal{O}(|V| \cdot |E|)$.

Przykład 5.6 (Działanie algorytmu Maximum Matching).

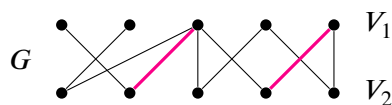
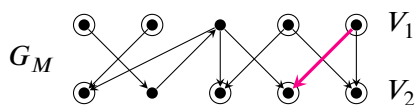
Prześledźmy działanie algorytmu MM na poniższym grafie G :



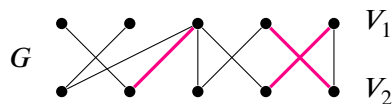
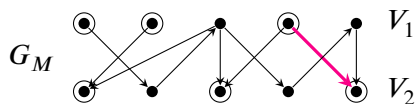
Iteracja 1:



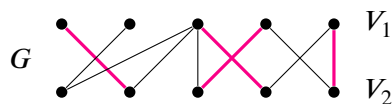
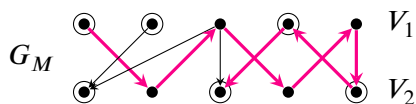
Iteracja 2:



Iteracja 3:



Iteracja 4:



Iteracja 5:

